



DEBRE BERHAN UNIVERSITY
COLLEGE OF INFORMATICS DEPARTMENT OF
INFORMATION TECHNOLOGY

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Maintenance Service Request and Tracking System (MSRTS) Prepared
for: Internship Software Project

Submitted by:

GROUP MEMBER	ID NUMBER
Takele Moges	1601139
Meseret Ayele	1600901
Sosina Tezera	1601117

Advisor: [Internship supervisors]

*Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor
of Science in Software Engineering*

**Debre Berhan, Ethiopia August
2026**

DECLARATION

We the undersigned, declare that this Software Requirements Specification (SRS), entitled “Maintenance Service Request and Tracking System (MSRTS)”, is our own original work, prepared under the guidance of our advisor, and has not been submitted, in whole or in part, for the award of any other degree, diploma, or certificate at this or any other institution. All sources of information used in this document have been duly acknowledged through in-text citations and a corresponding reference list, in accordance with the referencing style prescribed by the Department of Information Technology.

No.	Name	ID Number	Signature	Date
1	Takele Moges	1601139		04/12/2018 E.C
2	Meseret Ayele	1600901		04/12/2018 E.C
3	Sosina Tezera	1601117		04/12/2018 E.C

Approved by Advisor:

Name	Signature	Date
Internship supervisors		

ACKNOWLEDGMENT

We would like to express our sincere gratitude to our advisor, [Advisor Name], for the guidance, patience, and constructive feedback provided throughout the analysis and design of this project. We are also grateful to the staff of the Debre Berhan University facilities and ICT support offices who generously gave their time for interviews and shared insight into the current maintenance-reporting process.

We further thank the Department of Software Engineering, College of Informatics, for providing the guideline and academic environment that made this work possible, and our fellow students who participated in the questionnaire and usability discussions that shaped the requirements captured in this document. Finally, we thank our families and friends for their continued encouragement throughout this project.

ABSTRACT

Maintenance operations at Debre Berhan University, like those at many higher education institutions, are currently coordinated through informal channels: phone calls, verbal reports, and, at best, a hand-written log book kept in the facilities office. This approach leaves users unable to track the status of a reported problem, gives maintenance admins no reliable visibility into technician workload, and leaves management without the historical data needed to plan preventive maintenance or evaluate service performance.

This document presents the Software Requirements Specification (SRS) for the Maintenance Service Request and Tracking System (MSRTS), a proposed web-based application that digitizes the full life cycle of a maintenance request, from submission by a user, through routing and assignment by an admin, to execution and closure by a technician. The specification captures the background and motivation for the project, a description of the existing manual system and its bottlenecks, the functional and non-functional requirements of the proposed system, the analysis-level object and behavioral models (use cases, class model, sequence and activity diagrams), and the system design, including software architecture, database design, API specification, security design, and deployment topology.

The proposed system is designed around a layered architecture with a React.js frontend (MERN stack), a Node.js/Express.js REST API, and a MongoDB NoSQL document database, secured with JWT-based authentication and role-based access control, and deployed using Docker containers to a cloud or campus hosting environment. The document follows an Agile/Scrum development methodology and the structure prescribed by the Debre Berhan University ICT/Software Project SRS Guideline, and is intended to serve as the authoritative reference for the design, implementation, and testing phases of the project.

Keywords: Maintenance Management, Service Request Tracking, Software Requirements Specification, Web Application, Agile Development, Role-Based Access Control, Debre Berhan University.

TABLE OF CONTENTS

DECLARATION	iii
ACKNOWLEDGMENT	iv
ABSTRACT	v
LIST OF FIGURES	ix
LIST OF TABLES	x
LIST OF ABBREVIATIONS	xi
CHAPTER ONE INTRODUCTION OF THE PROJECT	1
1.1 Introduction	1
1.2 Background Information of Debre Berhan University	2
1.2.1 Vision of DBU	2
1.2.2 Mission of DBU	2
1.2.3 Relevance of the Project to DBU's Vision and Mission	2
1.3 Background of the Project.....	3
1.4 Motivation	3
1.5 Statement of the Problem	4
1.6 Team collaboration and Roles	5
1.7 Objective of the Project.....	6
1.7.1 General Objective	6
1.7.2 Specific Objectives	6
1.8 Scope and Limitation of the Project	7
1.9 Significance of the Project	8
1.10 Target Beneficiaries of the System	9
1.11 Methodology of the Project	9
1.11.1 Data Collection Methods	9
1.11.2 Development Methodology	10
1.11.3 System Analysis and Design Tools	10
1.11.4 Development Environment and Tech Stack	10
1.11.5 Testing Strategy	11
1.11.6 Project Management and Communication Tools	11
1.12 Risks and Contingencies	12

1.13 Time Plan	13
1.14 Budget Plan	16
1.15 Feasibility Study	17
1.15.1 Technical Feasibility	17
1.15.2 Economic Feasibility	17
1.15.3 Operational Feasibility	18
1.15.4 Schedule Feasibility	18
1.16 Assumptions and Dependencies	18
CHAPTER TWO DESCRIPTION OF THE EXISTING SYSTEM	20
2.1 Introduction to the Existing System	20
2.2 Actors / Stakeholders in the Existing System	20
2.3 Major Functions of the Existing System	20
2.4 Business Rules	21
2.5 Reports Generated in the Existing System	21
2.6 Forms and Documents of the Existing System	21
2.7 Bottlenecks of the Existing System	21
2.7.1 Performance	22
2.7.2 Security	22
2.7.3 Scalability	22
2.7.4 User Experience	22
2.8 Practices to Be Preserved	23
2.9 Proposed Solution for the New System	23
2.10 Requirements of the Proposed System	23
2.10.1 Functional Requirements	24
2.10.2 Non-Functional Requirements	26
CHAPTER THREE: SYSTEM ANALYSIS	29
3.1 System Models	29
3.1.1 Scenarios / User Stories	29
3.1.2 Use Case Model	30
3.1.3 Use Case Descriptions	32
3.1.4 Object Model	35
3.2 Dynamic Model	36
3.2.1 Sequence Diagrams	36
3.2.2 Activity / State Diagram	38
3.2.3 User Interface Prototype / Wireframes	39
CHAPTER FOUR : SYSTEM DESIGN	43
4.1 Introduction	43
4.1.1 Purpose of the System	43
4.1.2 Design Goals	43
4.2 Current Software Architecture	43

4.3 Proposed Software Architecture	44
4.3.1 Overview	44
4.3.2 Architectural Pattern	44
4.3.3 Subsystem Decomposition	45
4.3.4 API Design and Documentation	45
4.3.5 Hardware / Software Mapping	46
4.3.6 Persistent Data Management	47
4.3.7 Access Control and Security	48
4.3.8 Global Software Control	49
4.3.9 Boundary Conditions	49
4.4 User Interface Design Guidelines	50
4.5 Component Diagram	51
4.6 Deployment Diagram	52
4.7 Performance and Scalability Considerations	53
4.8 Backup and Disaster Recovery	54
CHAPTER FIVE :TESTING AND QUALITY ASSURANCE	55
5.1 Testing Objectives	55
5.2 Levels of Testing	55
5.2.1 Unit Testing	55
5.2.2 Integration Testing	55
5.2.3 End-to-End (E2E) Testing	55
5.2.4 User Acceptance Testing (UAT)	55
5.3 Test Case Design	56
5.4 Test Environment	58
5.5 Defect Management Process	58
CHAPTER SIX : CONCLUSION AND FUTURE WORK	59
6.1 Summary	59
6.2 Limitations of the Current Version	59
6.3 Future Enhancements	59
REFERENCES	61
APPENDIX A SAMPLE INTERVIEW QUESTIONS	62
A.1 Questions for Facilities / Maintenance Office Staff	62
A.2 Questions for Users (Students and Staff)	62
APPENDIX B SAMPLE QUESTIONNAIRE	63
APPENDIX C SAMPLE API REQUEST / RESPONSE PAYLOADS	64
C.1 POST /requests (Submit Maintenance Request --- FR-2, FR-3)	64
C.2 POST /requests/{id}/assign (Assign Technician --- FR-10)	64
APPENDIX D GLOSSARY OF TERMS	66
APPENDIX E REQUIREMENT TRACEABILITY MATRIX	. 67

LIST OF FIGURES

Figure	Title	Page
Figure 1.1	Indicative Project Time Plan (Gantt Chart)	15
Figure 3.1	Use Case Diagram of the Maintenance Service Request and Tracking System	32
Figure 3.2	Analysis-Level Class Diagram of MSRTS	33
Figure 3.3	Sequence Diagram — Submit Maintenance Request	37
Figure 3.4	Sequence Diagram — Assign Technician and Update Status	38
Figure 3.5	State Machine / Activity Diagram — Maintenance Request Lifecycle	39
Figure 3.6	Wireframe — User Dashboard	41
Figure 3.7	Wireframe — Admin Assignment Board (Kanban View)	41
Figure 4.1	Proposed Layered Software Architecture of MSRTS	44
Figure 4.2	Document Schema Diagram of MSRTS	48
Figure 4.3	Component Diagram of MSRTS	52
Figure 4.4	Deployment Diagram of MSRTS	53

Table	Title	Page
Table 1.1	Team Collaboration and Roles	6
Table 1.2	Development Environment and Tech Stack	10
Table 1.3	Risks and Contingencies	12
Table 1.4	Four-Week Daily Development Plan	13
Table 1.5	Budget Plan	16
Table 1.6	Feasibility Assessment Summary	17
Table 2.1	Actors / Stakeholders in the Existing System	20
Table 2.2	Functional Requirements Summary	26
Table 2.3	Non-Functional Requirements Summary	28
Table 3.1	Data Dictionary — Core Entities	36
Table 3.2	User Stories	30
Table 4.1	Subsystem Decomposition	45
Table 4.2	REST API Endpoint Summary	46
Table 4.3	Hardware / Software Mapping	47
Table 5.1	Sample Test Cases	58
Table E.1	Requirement Traceability Matrix	67

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
DBU	Debre Berhan University
FR	Functional Requirement
ICT	Information and Communication Technology
JWT	JSON Web Token
MSRTS	Maintenance Service Request and Tracking System
NFR	Non-Functional Requirement
ODM	Object Data Modeling
OOAD	Object-Oriented Analysis and Design
RBAC	Role-Based Access Control
REST	Representational State Transfer
SLA	Service-Level Agreement
SRS	Software Requirements Specification
TLS	Transport Layer Security
UAT	User Acceptance Testing
UI/UX	User Interface / User Experience
UML	Unified Modeling Language
WCAG	Web Content Accessibility Guidelines



CHAPTER ONE: INTRODUCTION OF THE PROJECT

1.1 Introduction

The Maintenance Service Request and Tracking System (MSRTS) is a web-based application designed to digitize the process by which staff, students, and departments of an institution report facility, equipment, and ICT maintenance problems, and by which maintenance teams receive, act on, and close those requests. In most Ethiopian higher-education institutions and comparable organizations, maintenance requests are still logged through phone calls, paper forms, or informal messages, which makes it difficult to track who reported what, who is responsible for fixing it, and how long a repair is taking. MSRTS replaces this ad-hoc process with a single, centralized platform where a request can be submitted in minutes, automatically routed to the right maintenance unit, tracked in real time by the user, and measured against service level targets by management.

This document is the Software Requirements Specification (SRS) for MSRTS. It describes the purpose, scope, functional and non-functional requirements, system models, and design of the proposed system, and it follows the structure prescribed by the Debre Berhan University ICT/Software Project SRS Guideline. The SRS is written for four audiences: the project team, who will use it as the authoritative reference during implementation; the project advisor and evaluation panel, who will use it to assess whether the proposed system adequately addresses the stated problem; the maintenance office and prospective end users, who can use it to confirm that their operational needs have been correctly captured; and future maintainers of the system, who will rely on this document, together with the source code and API documentation, to understand why the system is built the way it is.

The remainder of this chapter presents the background of the University and the project, the problem being solved, the objectives, scope, methodology, and project management plan. Chapter Two describes the existing manual system in detail. Chapter Three presents the system

analysis, including use cases and object and behavioral models. Chapter Four presents the proposed system design, covering architecture, database, API, security, and deployment.

1.2 Background Information of Debre Berhan University

Debre Berhan University (DBU) is a public university located in Debre Berhan, in the Amhara Regional State of Ethiopia, approximately 130 kilometers northeast of Addis Ababa. Established to expand access to higher education in the region, DBU offers undergraduate and postgraduate programs across several colleges, including the College of Informatics, College of Engineering, College of Natural and Computational Sciences, College of Business and Economics, College of Social Sciences and Humanities, College of Agriculture, and College of Health Sciences. The university maintains a growing number of campuses, lecture halls, computer laboratories, dormitories, staff residences, and administrative buildings, all of which require continuous maintenance of ICT equipment, electrical systems, plumbing, furniture, and civil infrastructure.

As the university's student population and physical footprint have grown, so has the volume and variety of maintenance demand across its campuses. This growth is the immediate operational context that motivates MSRTS: a manual, office-bound reporting process that may have been adequate for a smaller institution becomes a genuine bottleneck once the number of buildings, laboratories, and residents reaches the scale DBU now operates at.

1.2.1 Vision of DBU

To become one of the top ten research and academic universities in Ethiopia by producing competent graduates, problem-solving research, and community-oriented services that contribute to national development.



1.2.2 Mission of DBU

To provide quality education, conduct problem-solving research, and render community services through well-qualified staff, adequate infrastructure, and a conducive learning and working environment, while promoting good governance transparency, and continuous improvement across all its operations, including the facilities and services that support teaching, learning, and administration.

1.2.3 Relevance of the Project to DBU's Vision and Mission

MSRTS directly supports the university's mission of maintaining “adequate infrastructure and a conducive learning and working environment” by making it faster and more transparent to identify and resolve infrastructure faults. It also supports the mission's emphasis on “good governance, transparency, and continuous improvement” by introducing an auditable record of every maintenance request and a reporting layer that management can use to continuously improve facility operations,

1.3 Background of the Project

Every day, faculty offices, classrooms, laboratories and administrative units at DBU experience maintenance problems, ranging from broken projectors and non-functional computers to leaking pipes. Currently, such issues are reported informally to the responsible office, often verbally or by phone, with no unique record created for the request. As a result, requests are frequently forgotten, duplicated, or resolved out of order, and there is no historical data that management can use to plan preventive maintenance, allocate technicians, or evaluate the performance of the maintenance unit.

The MSRTS project was proposed to address this operational gap by giving the university a dedicated, auditable, and measurable maintenance-request workflow. The project originated from repeated, informal feedback gathered by the project team from students and staff

over the course of a semester, and was subsequently validated through structured interviews and a short questionnaire, described in Section 1.11.1, which confirmed that the pain points observed anecdotally were shared broadly across departments rather than being isolated to a single office.

1.4 Motivation

The team was motivated by first-hand observation of the delays and miscommunication that occur when maintenance issues are reported informally. Recurring complaints from students and staff about slow-to-repair projectors, unreliable Wi-Fi hardware, and Ict maintenance: a lightweight, easy-to-use tracking system could substantially shorten response times, increase accountability of maintenance staff, and give admins visibility they currently lack.

Building this system is also an opportunity to apply object-oriented analysis and design, modern web-application architecture, and agile development practices to a real institutional problem, and to produce a system that could plausibly be adopted by the university after graduation, rather than remaining a purely academic exercise. This dual motivation, solving a genuine operational problem while demonstrating the full software-engineering life cycle, shaped both the scope decisions in Section 1.8 and the emphasis on measurable outcomes (SLA compliance, turnaround time) throughout the requirements in Chapter Two.

1.5 Statement of the Problem

The existing manual process for handling maintenance requests at DBU, and in comparable organizations, suffers from the following problems:

- ❖ Requests are reported verbally, by phone, or on paper, so there is no single, searchable record of what was reported, by whom, and when.
- ❖ There is no way for a user to check the status of a previously reported problem without calling or visiting the maintenance office again.



Team Member	Role	Week 1 Tasks	Week 2 Tasks	Week 3 Tasks	Week 4 Tasks
Meseret Ayele	Backend Developer	System Architecture & Authentication Logic Setup (RBAC, Password Hashing)	Request Submission & Ticket Routing API Endpoints Development	Status Update, Resolution & Report Generation Backend Logic	API Integration Testing, Server Logic Optimize & Bug Fixes
Sosina Tezera	Frontend Developer	UI/UX Wire framing & Layout Structure Design	User Login Page & Digital Request Form Frontend Implementation	Technician Dashboard & Real-Time Ticket Tracking Interfaces	UI Responsiveness, Browser Testing & User Experience Refinement
Takele Moges	Database Developer & Analyst	Requirement Analysis	Relational Database Setup (MongoDB) & Table Normalization	Data Analytics Views Creation	Database Performance, Data Security Verify Backup Config

1.6 Team collaboration and Roles

Table 1.1: Team Collaboration and Roles

1.7 Objective of the Project

1.7.1 General Objective

The general objective of this project is to design and develop a web-based Maintenance Service Request and Tracking System that allows staff and students to submit maintenance requests online, allows maintenance technicians and admins to manage and track those requests, and provides admins with reporting tools to monitor performance.

1.7.2 Specific Objectives

- ❖ To study and document the existing manual maintenance-reporting process and identify its shortcomings.
- ❖ To design a NoSQL document database that models users, requests, categories, assets, technicians, assignments, and feedback.
- ❖ To develop a secure authentication and role-based authorization module for users, technicians, and admins.
- ❖ To develop a request-submission module that allows attachment of photos, category selection, priority setting, and location tagging.
- ❖ To develop an assignment and workflow module that lets admins assign requests to technicians and lets technicians update status.
- ❖ To develop a real-time status-tracking and notification module for users and technicians.
- ❖ To develop a reporting and analytics module that summarizes turnaround time, request volume by category, and technician performance.
- ❖ To design and document a Service-Level Agreement (SLA) mechanism that flags overdue requests to admins.
- ❖ To test the developed system for functionality, usability, and performance, and to deploy it in a cloud or campus hosting environment.



1.8 Scope and Limitation of the Project

The scope of MSRTS covers the full life cycle of a maintenance request: submission by a user, routing and assignment by an admin, execution and status update by a technician, and closure with optional feedback from the user. The system will support role-based dashboards for users, technicians, and admins; email/in-app notifications; a searchable request history; an asset register; and basic analytics/reporting. The system will be delivered as a responsive web application accessible on desktop and mobile browsers.

In Scope

- ❖ User registration, authentication, and role-based access control for User, Technician, and Admin roles.
- ❖ Submission, categorization, prioritization, and location-tagging of maintenance requests, with optional photo attachment.
- ❖ Manual assignment of technicians by an admin, informed by category match and current workload.
- ❖ Status tracking through a defined lifecycle (Submitted, Assigned, In Progress, Blocked, Completed, Closed, Rejected).
- ❖ In-app and email notifications on status changes.
- ❖ User feedback and rating on completed requests.
- ❖ SLA configuration per category and overdue-request alerts.
- ❖ Reporting dashboard with export to PDF/CSV.
- ❖ An asset register linking requests to specific pieces of equipment or infrastructure.

Out of Scope (Current Version) /limitations

- ❖ Integration with third-party enterprise resource planning (ERP) or financial systems for procurement of spare parts.

- ❖ Native mobile applications (iOS/Android); the system will instead be responsive/mobile-friendly through the browser.
- ❖ Automated IoT-based fault detection (e.g., sensors that automatically create a request); requests are user-initiated.
- ❖ Offline functionality; the system requires an active internet connection.
- ❖ Multi-language support beyond English in the initial release.
- ❖ Automatic (algorithmic) technician assignment; assignment is admin-driven in this version, with automation identified as future work in Section 6.3.

1.9 Significance of the Project

For the university, MSRTS will shorten the time between a fault being reported and being resolved, increase accountability of the maintenance unit through auditable records, and provide management with data to plan preventive maintenance and staffing. For students and staff, the system removes the friction of reporting a problem and gives visibility into its progress without repeated follow-up calls. For maintenance technicians, a clear, prioritized task list reduces wasted trips and duplicated effort. For the project team, the system is an opportunity to apply the full software engineering life cycle, from requirements elicitation through design, implementation, testing, and deployment, to a problem with genuine institutional value, and to produce a portfolioquality artifact that demonstrates competence in modern web-application engineering.



1.10 Target Beneficiaries of the System

Beneficiary	How They Benefit
Students, faculty, and staff (Users)	Can report a problem from any device in minutes and track its status without repeat calls or visits.
Maintenance technicians	Receive a clear, prioritized task list and a simple interface to update status and add resolution notes.
Maintenance admins / admins	Gain visibility into the full request queue and technician workload, enabling fair, category-matched assignment.
University management	Can use aggregated reports to plan preventive maintenance, staffing, and budgeting.
Future project teams / the university's ICT unit	Inherit a documented, extensible system rather than starting a digitization effort from scratch.

1.11 Methodology of the Project

1.11.1 Data Collection Methods

Requirements for this project were gathered using a combination of the following datacollection techniques:

- ❖ Interviews with maintenance office staff and a sample of users (students and administrative staff) to understand the current process and its pain points.
- ❖ A short questionnaire distributed to staff and students to quantify how often maintenance problems are reported and how satisfied they are with current response times.
- ❖ Document review of any existing maintenance log books, memos, or spreadsheets used by the facilities office.
- ❖ Direct observation of how a maintenance request currently moves from a user to a technician.

Sample interview questions and the questionnaire instrument used during requirements elicitation are provided in Appendix A and Appendix B respectively, so that the elicitation process can be reviewed or repeated by future teams.

1.11.2 Development Methodology

The project will follow an Agile/Scrum development methodology, delivered in short, iterative sprints (typically one to two weeks) rather than a single-pass Waterfall model. Agile/Scrum was chosen because the requirements of a maintenance-tracking workflow are best refined through repeated feedback from real stakeholders (users, technicians, and admins); each sprint produces a working increment (for example, request submission, then assignment, then notifications, then reporting) that can be demonstrated and adjusted before the next increment is built.

This iterative approach also fits naturally with Object-Oriented Analysis and Design (OOAD): each sprint refines the analysis-level class model, use cases, and sequence diagrams introduced in Chapter Three, so the



object model evolves alongside the working software instead of being frozen upfront as pure Waterfall would require.

A pure Waterfall approach was considered and rejected for this project because the team, acting as its own primary source of some requirements interpretation, has limited direct authority to mandate process change at the university; an iterative approach allows early increments to be validated with actual facilities-office staff before the full system is built, reducing the risk of building the wrong workflow.

1.11.3 System Analysis and Design Tools

- ❖ draw.io (diagrams.net) — use case, class, sequence, activity, and deployment diagrams.
- ❖ Figma — user interface wireframes and interactive prototypes.
- ❖ Lucidchart / Enterprise Architect (optional) — supplementary UML modeling and document schema diagrams.
- ❖ Graphviz / dbdiagram.io (optional) — automated regeneration of the ER and architecture diagrams directly from the schema as it evolves.

1.11.4 Development Environment and Technological Stack

Layer	Technology
Frontend	React.js and TypeScript, Tailwind CSS
Backend	Node.js with Express.js (REST API)
Database	MongoDB (primary NoSQL document store)
API Style	RESTful API, documented with OpenAPI/Swagger
Version Control	Git and GitHub, using a Git Flow branching strategy
Authentication	JWT-based authentication with Role-Based Access Control (RBAC)
Hosting/Deployment	Docker containers deployed to a cloud/free-tier host (e.g., Render, Vercel for frontend)
Project Management	Trello or Jira for the sprint backlog; Slack or Telegram for team communication



1.11.5 Testing Strategy

The system will be tested at multiple levels to ensure correctness and reliability. Chapter Five expands this into a full test plan with sample test cases; at a summary level, the strategy comprises:

- ❖ Unit testing of individual functions and components using Jest (and PyTest if any Python microservice is used).
- ❖ Integration testing of API endpoints using Postman/Newman to verify request/response contracts.
- ❖ End-to-end testing of critical user journeys (submit request, assign technician, update status, view report) using an automated E2E tool.
- ❖ User Acceptance Testing (UAT) with a small group of real users and maintenance staff before final deployment.

1.11.6 Project Management and Communication Tools

Sprint planning, backlog grooming, and progress tracking will be managed using a lightweight Kanban board (Trello) or Jira, mirroring the Admin Assignment Board concept the team is itself building into MSRTS. Daily coordination will happen over a shared Slack or Telegram channel, and weekly check-ins with the advisor will review the sprint's working increment against the plan in Section 1.13. All source code, decisions, and this document are version-controlled in a shared GitHub repository so that document history is auditable in the same way the deployed system is designed to make maintenance history auditable.

1.12 Risks and Contingencies

Risk	Likelihood	Impact	Mitigation / Contingency
Underestimated development time for notification module	Medium	Medium	Time-box the feature; fall back to in-app notifications only if email/SMS integration is delayed
Limited access to real maintenance-office data for testing	Medium	Low	Use realistic seed/sample data generated from interview findings
Team member unavailability	Low	Medium	Cross-train members on both frontend and backend tasks; maintain shared documentation
Hosting/infrastructure limits on free-tier cloud services	Medium	Low	Use lightweight containers and a managed free-tier database; monitor usage
Low adoption / resistance to change from facilities-office staff	Medium	High	Involve facilities staff in UAT early (Section 1.11.1); keep the submission flow simpler than the process it replaces
Data loss due to inadequate backup	Low	High	Automated daily database backups (Section 4.8); tested



1.13 Time Plan

Table 1.4 presents a day-by-day breakdown of the project across four accelerated development weeks (28 days), grouped by focus area; Figure 1.1 presents the same plan as a Gantt chart for quick visual reference. The plan follows an Agile/Scrum rhythm of weekly sprints, with the MERN/MongoDB stack implemented incrementally from environment setup through deployment and documentation.

Week	Day	Focus Area	Key Activities
Week 1	Day 1	Setup	Kickoff & environment setup: Node.js/Express scaffold, MongoDB Atlas cluster, Git repo, CI skeleton
Week 1	Day 2	Backend	Define Mongoose schemas & models (User, Request, Category, Department, Notification)
Week 1	Day 3	Backend	Implement authentication: JWT issuing, bcrypt/argon2 password hashing
Week 1	Day 4	Backend	Implement role-based access-control middleware and registration/login endpoints
Week 1	Day 5	Backend	Build Request-submission API (create/read requests, photo-upload handling)
Week 1	Day 6	Testing	Unit tests for authentication and request submission modules
Week 1	Day 7	Review	Sprint 1 review, buffer/catch-up, retrospective
Week 2	Day 8	Backend	Implement technician-assignment API (queue, filter by category and workload)



Week 2	Day 9	Backend	Implement status-update and status-history endpoints
Week 2	Day 10	Backend	Implement in-app and email notification service
Week 2	Day 11	Frontend	Scaffold React app: routing, layout, Tailwind CSS, authentication context
Week 2	Day 12	Frontend	Build User dashboard UI (submit request, track status)
Week 2	Day 13	Frontend	Build Technician dashboard UI (task list, status updates)
Week 2	Day 14	Testing	Integration testing of Sprint 1 & 2 modules; sprint review
Week 3	Day 15	Frontend	Build Admin dashboard UI (assignment queue, technician management)
Week 3	Day 16	Backend	Implement SLA-configuration API and MongoDB aggregation pipelines for reports
Week 3	Day 17	Frontend	Build Reports/Analytics UI (charts, filters, PDF/CSV export)
Week 3	Day 18	Backend	Implement account management (admin CRUD for users and categories)
Week 3	Day 19	Frontend	Accessibility (WCAG) and responsive design pass across all dashboards

Week 3	Day 20	Security	Security hardening: rate limiting, input validation, NoSQL-injection prevention
Week 3	Day 21	Review	Sprint 3 review and bug triage
Week 4	Day 22	Testing	Complete unit and integration testing; review coverage

Week 4	Day 23	Testing	End-to-end (E2E) testing of critical user flows
Week 4	Day 24	Testing	User Acceptance Testing (UAT) with facilities staff and sample users
Week 4	Day 25	Testing	Bug fixing from UAT feedback
Week 4	Day 26	Deploy	Deployment: Docker build, CI/CD pipeline, MongoDB Atlas production configuration
Week 4	Day 27	Docs	Finalize documentation: SRS refinement, API docs, user manual
Week 4	Day 28	Docs	Final demo, project submission, retrospective

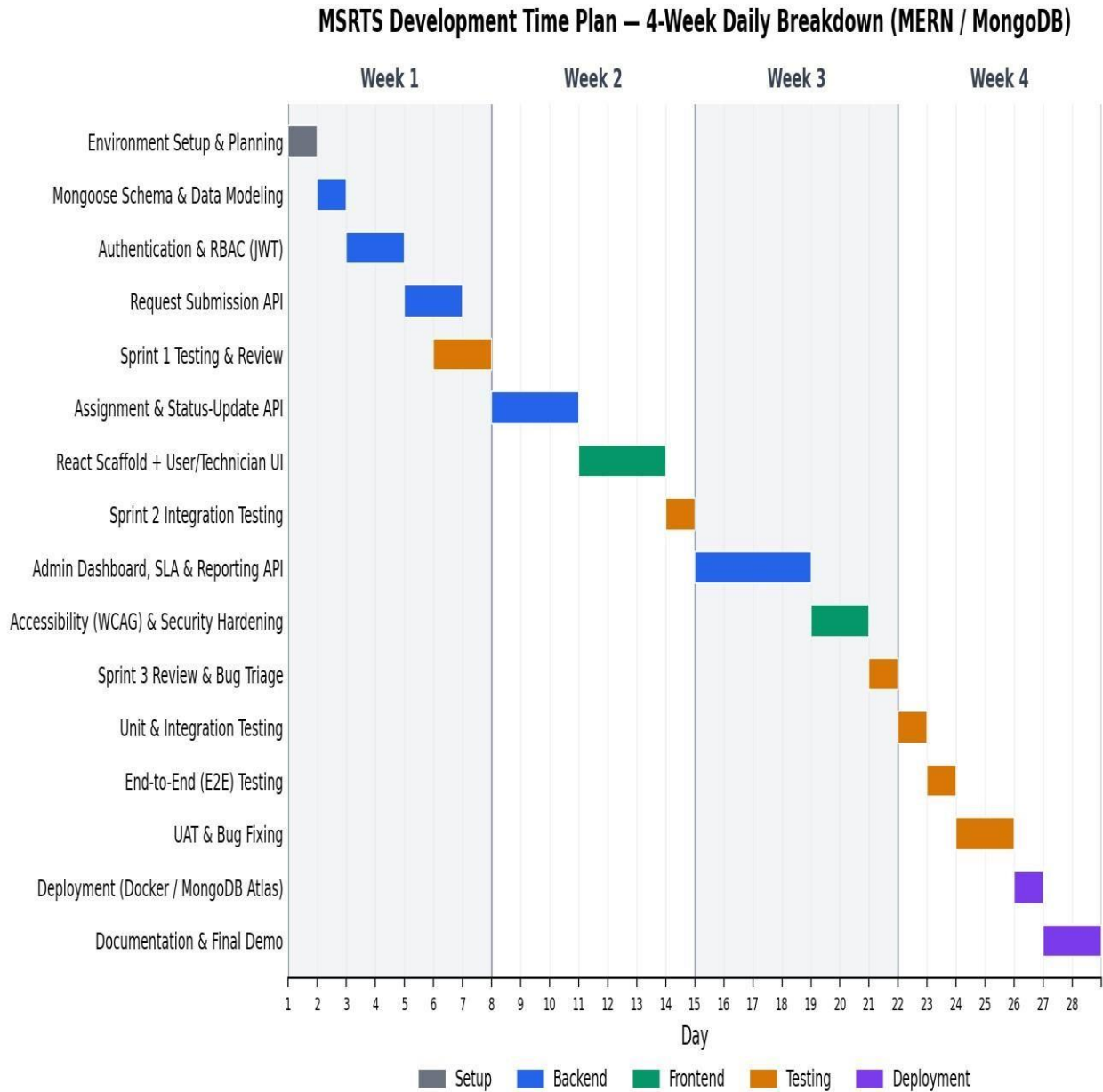


Figure 1.1: Indicative Project Time Plan (Gantt Chart)

1.14 Budget Plan

<i>No.</i>	<i>Budget Category / Description</i>	<i>Detail / Unit</i>	<i>Rate (ETB)</i>	<i>Total Cost (ETB)</i>
1	Hardware Equipment	Laptops/PCs & Test Mobile Devices (For testing UI responsiveness & API)	Lump sum	50,000 ETB
2	Development & Technical Labor	Developers / 1 Month	20,000/person	60,000 ETB
3	Transport & Mobility Allowance	Members / 4 Weeks (Local Travel for Requirements & Testing)	1,000 / person	3,000 ETB
4	Connectivity & Communication	Data & Internet Services (4 Weeks)	1,000 / package	3,000 ETB
5	Software Stack & Hosting Setup	Open Source Stack & Local DBU Server Setup	Lump sum	4,000 ETB
6	Documentation & Printing	Project Reports & Operational Manuals Printing	Lump sum	2,000 ETB
7	Contingency Fund	Emergency & Unexpected Costs (~10%)	Reserve	8,100 ETB
TOTAL	GRAND TOTAL PROJECT BUDGET			130,100 ETB



1.15 Feasibility Study

A brief, multi-dimensional feasibility assessment was carried out before committing to the project scope defined in Section 1.8. The assessment considered technical, economic, operational, and schedule feasibility, summarized in Table 1.6.

1.15.1 Technical Feasibility

All technologies proposed in the tech stack (Section 1.11.4) are open-source or available on free tiers, are well documented, and are already familiar to at least one member of the project team from prior coursework. The functional requirements in Section 2.10.1 do not require any capability (real-time updates, file upload, role-based access, aggregation-based reporting) that falls outside what a standard three-tier web application can provide, so the project is judged technically feasible within the team's existing skill set and the free-tier infrastructure identified in Section 1.11.4.

1.15.2 Economic Feasibility

The budget in Section 1.14 relies almost entirely on free-tier cloud services and existing student internet access, with the only material cost being an optional domain name; total estimated project cost is under 6,000 ETB. Because the system is intended as an internal tool rather than a commercial product, no revenue model is required; the return on investment is measured instead in reduced maintenance turnaround time and administrative overhead, both of which are inexpensive to demonstrate in a university deployment.

1.15.3 Operational Feasibility

Operational feasibility depends on whether facilities-office staff and users will actually use the system once built. This risk is explicitly tracked in Table 1.3 (“low adoption / resistance to change”) and mitigated by involving facilities staff in interviews (Section 1.11.1) and UAT (Section 1.11.5) from an early stage, and by keeping the user-facing submission flow (Figure 3.6) simpler than the phone-call process it replaces, so that the system reduces rather than adds friction for its least technical users.

1.15.4 Schedule Feasibility

The four-week, day-by-day plan in Section 1.13 is an accelerated schedule built around weekly Agile/Scrum sprints, matching the deadlines stated in the DBU SRS guideline. Each week targets a demonstrable set of modules rather than the whole system at once, which keeps the schedule resilient to the kind of minor slippage anticipated in the risk register.

Dimension	Assessment
Technical	Feasible — standard three-tier stack, no unfamiliar capability required
Economic	Feasible — under 6,000 ETB, free-tier infrastructure
Operational	Feasible with mitigation — requires early stakeholder buyin
Schedule	Feasible — fits the four-week accelerated sprint timeline



1.16 Assumptions and Dependencies

- ❖ Users, technicians, and admins /admin have access to a device with a modern web browser and a working internet connection.
- ❖ The facilities/ICT office is willing to designate at least one staff member to act as an Admin during UAT and after deployment.
- ❖ A free-tier or campus hosting environment (Section 4.6) is available for the duration of development and demonstration.
- ❖ Email delivery (for notifications) is available either through a free transactionalemail provider or the university's existing email infrastructure.
- ❖ The categorization scheme (electrical, plumbing, ICT, carpentry, general) identified during interviews is representative of the maintenance specialties the university actually staffs; any additional specialty can be added as a new Category record (Section 3.1.4.1) without a schema change.

CHAPTER TWO: DESCRIPTION OF THE EXISTING SYSTEM

2.1 Introduction to the Existing System

The existing maintenance-reporting process at DBU, and at comparable institutions, is a manual, paper- and phone-based workflow. There is no dedicated software system; instead, a user who experiences a problem, a broken chair, a non-working projector, a leaking tap, or a faulty electrical socket, reports it directly to the department secretary, the facilities office, or a technician they happen to know, either by phone, in person, or by writing a note. The facilities office keeps, at best, a

handwritten log book, and there is no shared, centralized, or searchable record of requests. This section describes that process in enough detail to justify the design decisions made in Section 2.9 and Chapter Four.

2.2 Actors / Stakeholders in the Existing System

Actor	Role in the Existing (Manual) System
User (student/staff)	Identifies a problem and reports it verbally, by phone, or on paper
Department secretary / office assistant	Receives the report and relays it to the facilities office
Facilities/maintenance office clerk	Logs the report (if at all) and informs an available technician
Maintenance technician	Travels to the location, diagnoses, and repairs the fault
Facilities admin	Oversees technicians informally, with little visibility into workload or backlog

Table 2.1: Actors / Stakeholders in the Existing System



2.3 Major Functions of the Existing System

- ❖ Verbal or written reporting of a maintenance problem to a nearby office.
Manual relay of the report from the user's office to the facilities/maintenance office.
- ❖ Informal, largely first-come-first-served assignment of a technician to the reported problem.
- ❖ Manual, in-person confirmation (if any) that the repair was completed.

2.4 Business Rules

- ❖ A maintenance request should, in principle, be attended to by a technician from the appropriate specialty (electrical, plumbing, ICT, carpentry, general).
- ❖ Urgent issues (e.g., electrical hazards, water leaks affecting equipment) are expected to be prioritized over routine issues, though this prioritization is informal and depends on who receives the report.
- ❖ Only facilities-office staff are expected to dispatch technicians; individual users are not supposed to contact technicians directly, although in practice this rule is often bypassed.

2.5 Reports Generated in the Existing System

The existing manual process produces little to no structured reporting. At most, some facilities offices maintain a hand-written log book listing the date, a short description, and, sometimes, the name of the technician who handled it. There is no monthly summary of request volume, no breakdown by category or building, and no measurement of average turnaround time. This absence of reporting is precisely the gap that the Reporting and Analytics functional requirements in Section 2.10.1 and the reporting subsystem in Section 4.3.3 are designed to close.

2.6 Forms and Documents of the Existing System

The only artifact typically used is an informal maintenance log book kept in the facilities office, and occasionally a handwritten memo or note passed from a

department to the facilities office. There is no standardized request form, no ticket number, and no receipt given to the user.

2.7 Bottlenecks of the Existing System

This section covers performance, security, scalability, and user-experience gaps specifically, since these four dimensions map directly onto the non-functional requirement categories defined in Section 2.10.2.

2.7.1 Performance

- ❖ Requests can take days to reach the correct technician because they are relayed verbally through multiple people.
- ❖ There is no way to measure or enforce turnaround time, so similar problems can be resolved in wildly different timeframes.
- ❖ Peak periods (start of semester, exam weeks) create request backlogs that are invisible to management until they become visible as complaints.

2.7.2 Security

- ❖ Because there is no authentication or access control, anyone can claim to have reported a problem, and there is no audit trail of who requested what and when.
- ❖ Sensitive information (e.g., which offices have broken locks) is only as secure as an unattended log book.
- ❖ There is no way to restrict who can mark a request as resolved, so completion status is effectively self-reported by whoever last touched the log book.

2.7.3 Scalability

- ❖ The manual process does not scale as the number of buildings, students, and staff grows; the facilities office becomes a bottleneck during peak periods (e.g., start of semester).
- ❖ Adding a new building, department, or maintenance specialty requires informal, word-of-mouth changes to who-reports-to-whom, rather than a configuration change.



2.7.4 User Experience

- ❖ Users have no way to check status without physically visiting or calling the facilities office again, which discourages reporting minor-but-important issues.
Technicians have no prioritized task list and often learn of new requests informally, leading to duplicated trips and missed requests.
- ❖ There is no feedback loop: a user whose problem is resolved poorly has no structured way to flag that, so quality issues are invisible to admins.

2.8 Practices to Be Preserved

- ❖ Routing requests by specialty (electrical, plumbing, ICT, carpentry, general), this categorization logic is sound and will be formalized, not discarded.
- ❖ Prioritizing urgent/safety-related issues, this practice will be preserved and made explicit through a priority field.
- ❖ Direct, low-friction reporting close to the point of the problem, MSRTS keeps this by allowing requests to be submitted from any device, without requiring an in-person visit to an office.

2.9 Proposed Solution for the New System

MSRTS proposes to replace the manual, undocumented workflow with a web-based platform where every request is captured as a structured record with a unique ID, category, priority, location, and status. Users submit requests online and can track them at any time; admins see an organized queue and can assign technicians based on specialty and workload; technicians receive a clear task list and update status as they work; and admins can generate reports on volume, turnaround time, and recurring problems.

This directly targets the bottlenecks identified in Section 2.7: performance improves because routing is automatic rather than relayed through multiple people, security improves because every action is authenticated and logged, scalability improves because the system is software rather than a single physical log book, and user experience improves because users and technicians both have real-time visibility into request status. Table 2.2 and Table 2.3 below translate this proposed solution into concrete, verifiable requirements.

2.10 Requirements of the Proposed System

2.10.1 Functional Requirements

The proposed system shall provide the following functions, grouped by actor. Each requirement is assigned an identifier (FR-n) that is referenced from the use case descriptions in Section 3.1.3 and from the Requirement Traceability Matrix in Appendix E.

User Functions

- ❖ FR-1: The system shall allow a user to register and log in securely.
- ❖ FR-2: The system shall allow a user to submit a maintenance request with a category, description, location, priority, and optional photo attachment.
- ❖ FR-3: The system shall assign a unique tracking ID to every submitted request and confirm submission to the user.
- ❖ FR-4: The system shall allow a user to view the real-time status of their own requests (submitted, assigned, in progress, completed, closed, or rejected).
- ❖ FR-5: The system shall allow a user to receive notifications when the status of their request changes.
- ❖ FR-6: The system shall allow a user to rate and leave feedback on a completed request.

Technician Functions

- ❖ FR-7: The system shall allow a technician to view a list of requests assigned to them, sorted by priority and due date.
- ❖ FR-8: The system shall allow a technician to update the status of an assigned request and add resolution notes.
- ❖ FR-9: The system shall allow a technician to mark a request as completed, which triggers a notification to the user.



Admin Functions

- ❖ FR-10: The system shall allow an admin to view all incoming requests and assign a technician based on category and workload.



FR-11: The system shall allow an admin to manage user accounts, roles, categories, and departments.

FR-12: The system shall allow an admin to configure Service-Level Agreement (SLA) targets per category and receive alerts when a request approaches or exceeds its SLA.

❖ FR-13: The system shall allow an admin to generate reports on request volume, average turnaround time, category breakdown, and technician performance, exportable as PDF/CSV.

❖ FR-14: The system shall maintain an audit log of key actions (submission, assignment, status change, closure) for accountability.

Table 2.2 summarizes these requirements with an indicative priority (using a High / Medium / Low priority scale) and the elicitation source that motivated each one, to make clear that no requirement was invented without a traceable origin in Section 1.11.1's data-collection activities.

ID	Requirement (summary)	Priority	Source
FR-1	Secure registration and login	High	Security bottleneck, Sec. 2.7.2
FR-2	Submit request with category/priority/location/photo	High	Interviews, user pain points
FR-3	Unique tracking ID on submission	High	“no searchable record”, Sec. 1.5
FR-4	View real-time status of own requests	High	“no way to check status”, Sec. 1.5
FR-5	Status-change notifications	Medium	UX bottleneck, Sec. 2.7.4
FR-6	Rate / feedback on completed request	Medium	“no confirmation mechanism”, Sec. 1.5



FR-7	Technician task list, sorted by priority	High	UX bottleneck, Sec. 2.7.4
FR-8	Technician status update + notes	High	Reporting gap, Sec. 2.5
ID	Requirement (summary)	Priority	Source
FR-9	Completion triggers notification	Medium	UX bottleneck, Sec. 2.7.4
FR- 10	Admin assignment by category/workload	High	“no visibility into workload”, Sec. 1.5
FR- 11	Admin management of users/categories/departments	High	Scalability bottleneck, Sec. 2.7.3
FR- 12	SLA configuration and overdue alerts	Medium	“no data on turnaround time”, Sec. 1.5
FR- 13	Reports on volume/turnaround/performance	High	Reporting gap, Sec. 2.5
FR- 14	Audit log of key actions	Medium	Security bottleneck, Sec. 2.7.2

Table 2.2: Functional Requirements Summary

2.10.2 Non-Functional Requirements

Performance, Scalability, Availability

- ❖ NFR-1: The system shall return search and dashboard results within 2 seconds under normal load (up to 200 concurrent users).
- ❖ NFR-2: The system architecture shall support horizontal scaling of the API layer via containerized, stateless application servers.
- ❖ NFR-3: The system shall target 99% uptime during business hours, excluding scheduled maintenance windows.

Security

- ❖ NFR-4: The system shall enforce authentication for all non-public endpoints using JWT-based sessions.
- ❖ NFR-5: The system shall enforce Role-Based Access Control (RBAC) so that users, technicians, and admins can only perform actions permitted for their role.
- NFR-6: The system shall store passwords using a salted hash (bcrypt or argon2) and never store plain-text passwords.
- ❖ NFR-7: The system shall transmit all data over HTTPS/TLS and validate/sanitize all user input to mitigate injection attacks.

☐ **Accessibility (WCAG)**

- ☐ NFR-8: The user interface shall conform to WCAG 2.1 Level AA guidelines, including sufficient color contrast, keyboard navigability, and screenreaderfriendly labels.

Maintainability and Responsiveness

- ❖ NFR-9: The codebase shall follow a layered/clean architecture (Section 4.3) with documented REST APIs (OpenAPI/Swagger) to ease future maintenance.
- ❖ NFR-10: The user interface shall be responsive and usable on screen widths from 360px (mobile) to 1920px (desktop).

**Usability**

- ❖ NFR-11: A first-time user shall be able to submit a request without training, in five steps or fewer from login.

**Reliability and Data Integrity**

- ❖ NFR-13: The database shall enforce referential integrity between requests, categories, assets, and users through foreign-key constraints (Section 4.3.6.1).

Table 2.3 groups these fourteen non-functional requirements by category for quick reference during design review.

Category	Requirement IDs	Design Sections That Address It
Performance, Scalability, Availability	NFR-1, NFR-2, NFR-3	4.1.2, 4.3.2, 4.6, 4.7
Security	NFR-4, NFR-5, NFR-6, NFR-7	4.3.7
Accessibility	NFR-8	4.4
Maintainability & Responsiveness	NFR-9, NFR-10	4.3.2, 4.3.4, 4.4
Usability	NFR-11, NFR-12	3.2.3, 4.4
Reliability & Data Integrity	NFR-13, NFR-14	4.3.6.1, 4.8

Table 2.3: Non-Functional Requirements Summary

CHAPTER THREE: SYSTEM ANALYSIS

3.1 System Models

This chapter presents the analysis-level models of MSRTS: the scenarios and user stories that motivated the functional requirements in Section 2.10.1, the use case model and detailed use case descriptions, the analysis-level object model, and the dynamic (behavioral) models, sequence diagrams, an activity/state diagram, and the user-interface prototype.

3.1.1 Scenarios / User Stories

The following user stories were derived from stakeholder interviews and guide the functional requirements in Section 2.10.1. Table 3.2 assigns each story an identifier and an indicative priority for traceability

ID	User Story	Priority
US-1	As a user, I want to submit a maintenance request with a photo, so that the technician understands the problem before arriving.	High
US-2	As a user, I want to see the status of my request online, so that I do not need to call the facilities office repeatedly.	High
US-3	As an admin, I want to see all unassigned requests sorted by priority, so that urgent problems are handled first.	High
US-4	As a technician, I want a task list of my assigned requests, so that I can plan my day efficiently.	High
US-5	As an admin, I want a monthly report of turnaround times by category, so that I can identify recurring problems and plan preventive maintenance.	Medium

US-6	As a user, I want to rate the completed repair, so that admins can monitor technician performance.	Medium
US-7	As an admin, I want to be alerted when a request is close to breaching its SLA, so that I can re-prioritize before it becomes overdue.	Medium
US-8	As an admin, I want to add a new maintenance category, so that the system reflects new specialties without a code change.	Low

Table 3.2: User Stories

3.1.2 Use Case Model

Figure 3.1 shows the use case diagram for MSRTS, covering three primary actors: the User, the Maintenance Technician, and the Admin (who also performs the account- and category-management functions of a System Admin). The Submit Maintenance Request and Assign Technician use cases include the Register/Login use case, since both require an authenticated session; both Assign Technician and Update Request Status include Receive Notifications,

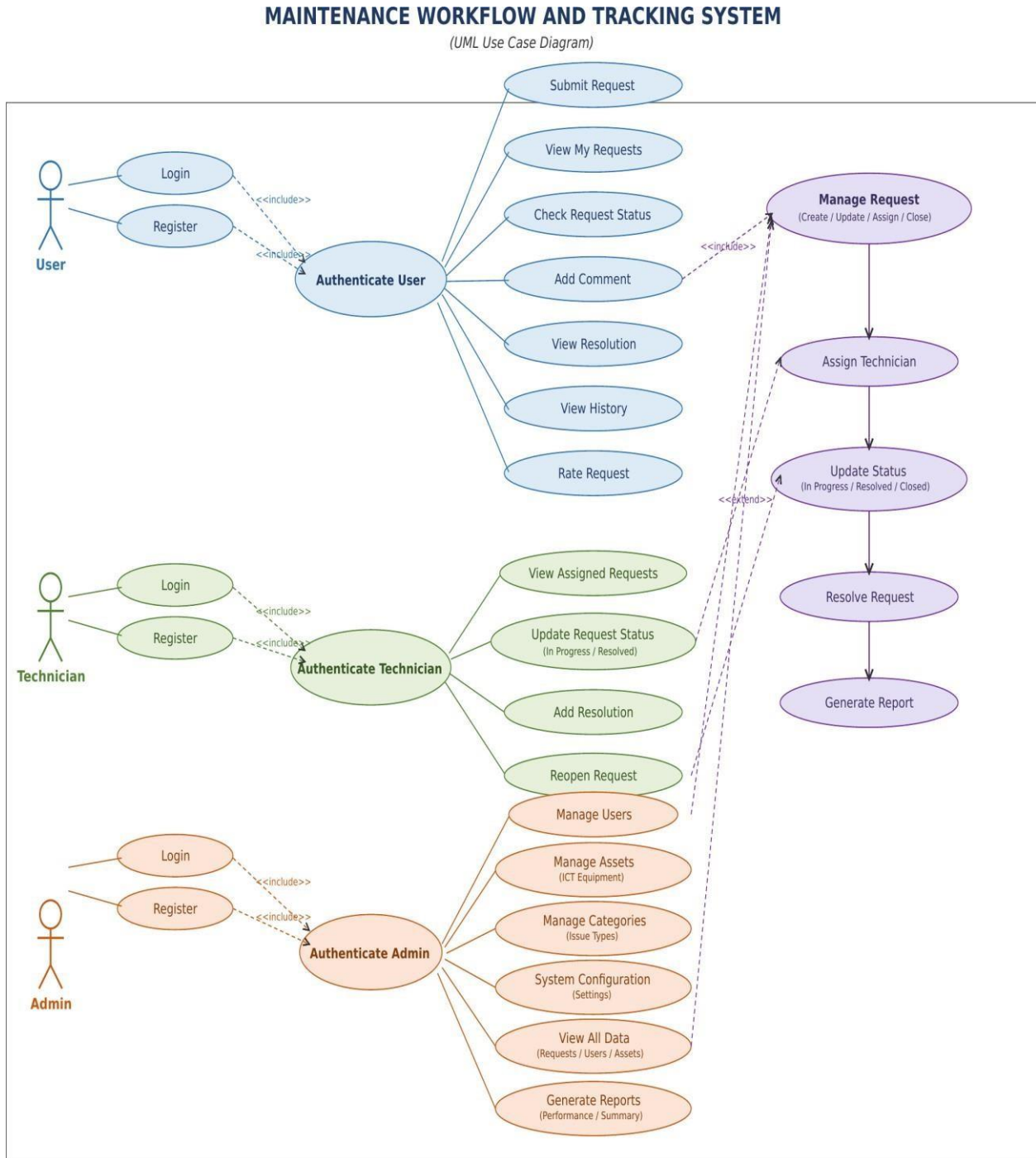


Figure 3.1: Use Case Diagram of the Maintenance Service Request and Tracking System

3.1.3 Use Case Descriptions

□ Use Case: Submit Maintenance Request

Use Case ID	UC-1
Actor	User
Related Requirements	FR-1, FR-2, FR-3
Precondition	User is registered and logged in
Main Flow	1) User selects 'New Request'. 2) User selects category, location, and priority, and enters a description. 3) User optionally attaches a photo. 4) User submits the form. 5) System validates input, creates a request record with status 'Submitted', assigns a unique tracking ID, and notifies the user.
Alternative Flow	If required fields are missing, the system displays a validation error and the request is not created.
Exception Flow	If the photo upload exceeds the maximum allowed size, the system rejects the file and prompts the user to attach a smaller image, without discarding the rest of the form.
Postcondition	A new request exists with status 'Submitted' and is visible in the admin's assignment queue.

□ Use Case: Assign Technician to Request

Use Case ID	UC-2
Actor	Admin
Related Requirements	FR-10
Precondition	Request exists with status 'Submitted' and admin is logged in
Main Flow	1) Admin opens the unassigned-requests queue. 2) Admin selects a request. 3) System suggests technicians filtered by category and current workload. 4) Admin selects a technician
	and confirms. 5) System sets status to 'Assigned', records the assignment, and notifies the technician and user.
Alternative Flow	If no technician of the matching specialty is available, the admin may reassign the category or escalate manually.
Exception Flow	If the selected technician is deactivated between page-load and confirmation, the system re-displays the technician list with an error rather than creating an invalid assignment.
Postcondition	Request status changes to 'Assigned' and appears in the technician's task list.

➤ Use Case: Update Request Status

Use Case ID	UC-3
Actor	Maintenance Technician
Related Requirements	FR-7, FR-8, FR-9
Precondition	Request is assigned to the logged-in technician
Main Flow	1) Technician opens an assigned request. 2) Technician updates status (e.g., 'In Progress', 'Completed') and adds resolution notes. 3) System timestamps the change and notifies the user.
Alternative Flow	Technician may flag the request as 'Blocked' (e.g., waiting for spare parts), which notifies the admin.
Exception Flow	If the technician attempts to set status directly from 'Assigned' to 'Completed' without an 'In Progress' step, the system allows it but logs the transition for admin review.
Postcondition	Request status and history are updated; user sees the change in real time.

➤ **Use Case: Generate Reports & Analytics**

Use Case ID	UC-4
Actor	Admin
Related Requirements	FR-13
Precondition	Admin is logged in
Main Flow	1) Admin selects a reporting period and filters (category, department, technician). 2) System aggregates request data (volume, average turnaround, SLA compliance). 3) Admin views the dashboard or exports the report as PDF/CSV.
Alternative Flow	If no data exists for the selected period, the system displays an empty-state message.
Exception Flow	If the export request times out for a very large date range, the system offers to email the completed export instead of blocking the dashboard.
Postcondition	Admin obtains a report usable for planning and performance review.

➤ **Use Case: Configure SLA Targets**

Use Case ID	UC-5
Actor	Admin
Related Requirements	FR-12
Precondition	Admin is logged in and at least one category exists
Main Flow	1) Admin opens SLA settings. 2) Admin sets a target resolution time (in hours) per category. 3) System stores the target and begins evaluating open requests against it. 4) System flags any request that has crossed 80% of its SLA window and alerts the assigned admin.
Alternative Flow	Admin may set different SLA targets per priority level within a category (e.g., urgent electrical vs. routine electrical).
Postcondition	New and existing open requests in that category are evaluated against the updated SLA target.

3.1.4 Object Model

3.1.4.1 Data Dictionary

Table 3.1 summarizes the core entities and attributes used throughout the analysis and design; the corresponding document schema diagram appears in Section 4.3.6.1 (Figure 4.2).

Entity	Key Attributes	Description
--------	----------------	-------------

User	user_id (PK), full_name, email, password_hash, role, department_id (FK)	Any person who can log in: user, technician, or admin
Department	department_id (PK), name, location	An academic or administrative unit of the university
ServiceRequest	request_id (PK), requester_id (FK), category_id (FK), asset_id (FK), description, priority, status, created_at, due_at	A single maintenance request from submission to closure
Category	category_id (PK), name, default_sla_hours	Type of maintenance issue (electrical, plumbing, ICT, carpentry, general)
Asset	asset_id (PK), asset_tag, name, department_id (FK)	A piece of equipment or fixture that can be the subject of a request
Assignment	assignment_id (PK), request_id (FK), technician_id (FK), assigned_at, completed_at	Links a request to the technician responsible for it
Feedback	feedback_id (PK), request_id (FK), rating, comment	User's rating and comment after a request is closed
Notification	notification_id (PK), user_id (FK), request_id (FK), message, sent_at	A message sent to a user about a change in request status

Table 3.1: Data Dictionary — Core Entities

3.1.4.2 Analysis-Level Class Diagram

At the analysis level, the object model mirrors the data dictionary above: a User, specialized as User, Technician, or Admin, is associated with zero or more ServiceRequest objects; each ServiceRequest belongs to one Category, may reference one Asset, and has zero or one Assignment linking it to a Technician; a closed ServiceRequest may have one Feedback; and status changes generate Notification objects sent to the relevant User. Figure 4.2 in Section 4.3.6.1 presents this same set of entities and relationships in document schema notation once implementation-level concerns, such as normalization and indexing, are addressed, so it is not duplicated here; the two views are intended to be read together, with this section describing the objects and their responsibilities, and Section 4.3.6.1 describing how they are persisted.

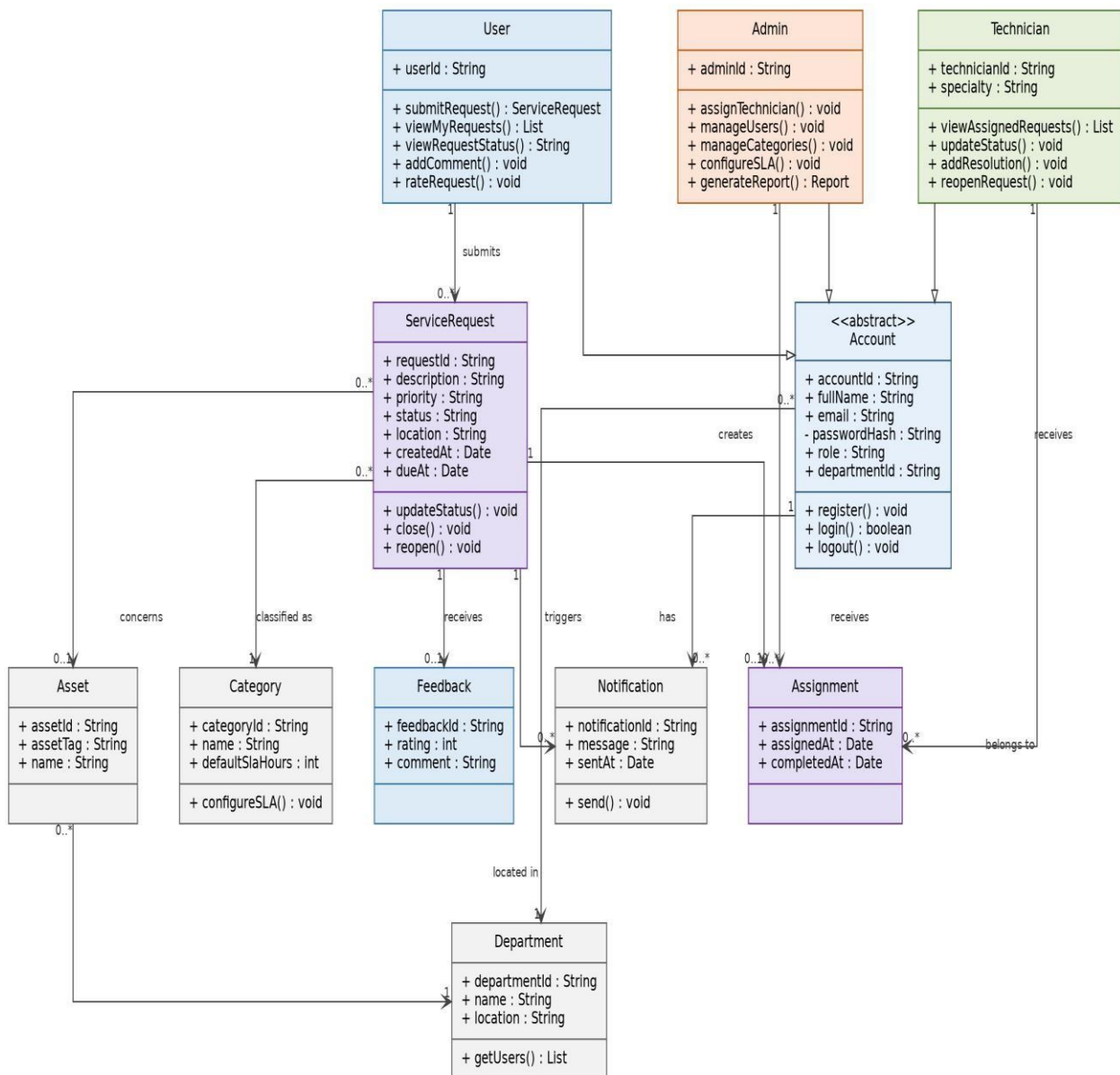


Figure 3.2: Analysis-Level Class Diagram of MSRTS

3.2 Dynamic Model

3.2.1 Sequence Diagrams

Figure 3.3 traces the Submit Maintenance Request use case (UC-1) from the user's form submission through to confirmation. Figure 3.4 traces the Assign Technician (UC-2) and Update Request Status (UC-3) use cases in a single diagram, since assignment and status update form one continuous admin-to-technician handoff.

Figure 3.2: Sequence Diagram — Submit Maintenance Request

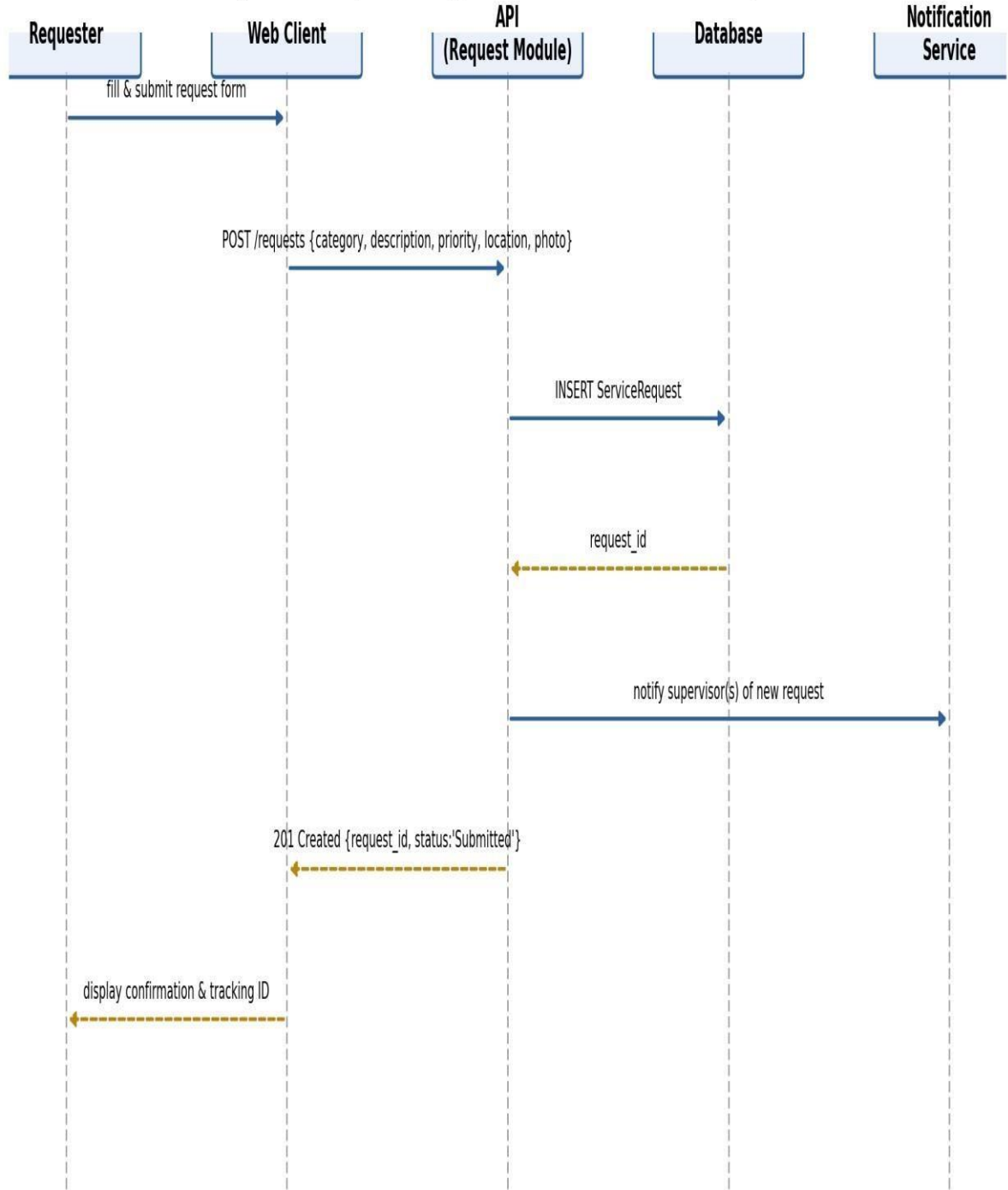


Figure 3.3: Sequence Diagram — Submit Maintenance Request

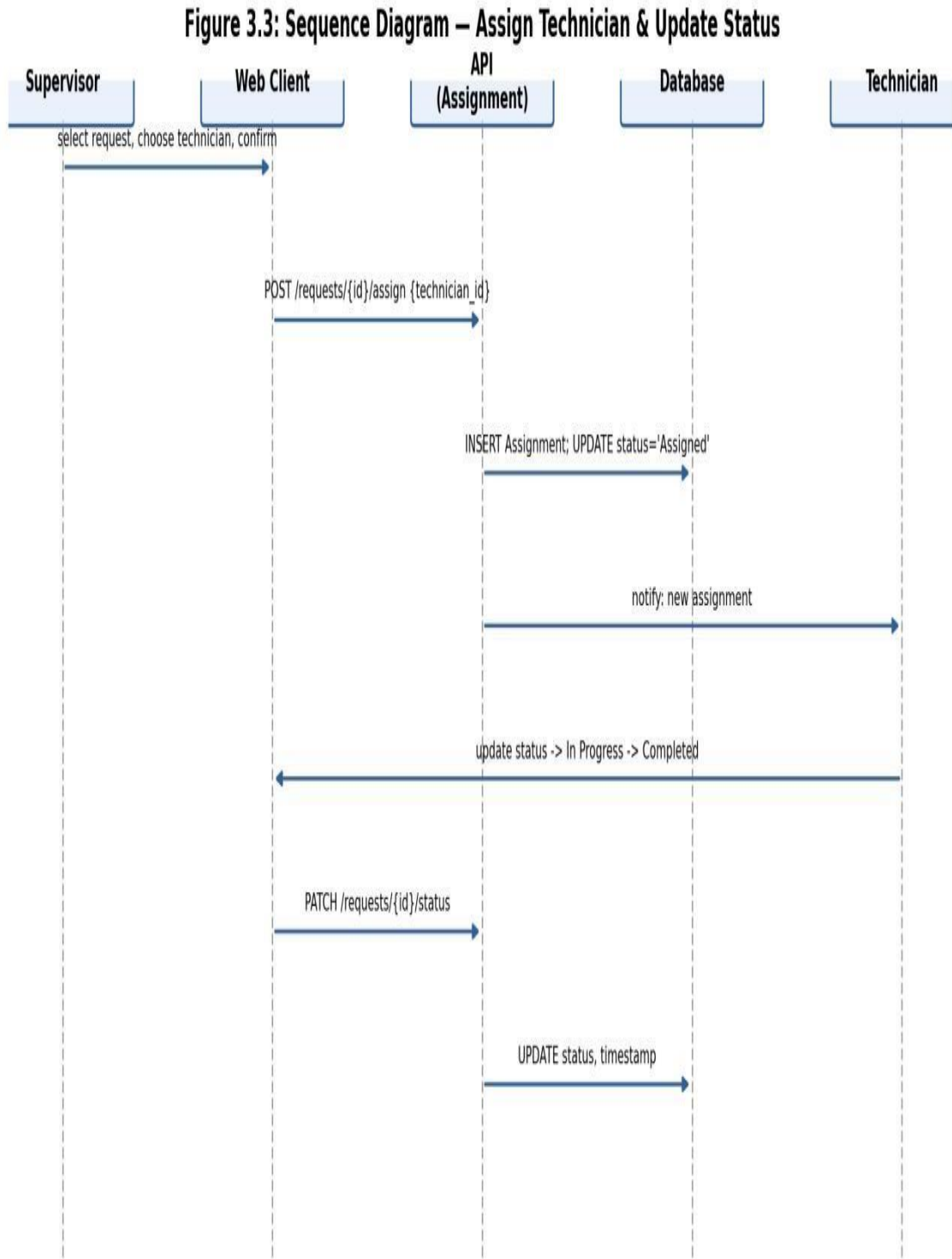


Figure 3.4: Sequence Diagram — Assign Technician and Update Status

3.2.2 Activity / State Diagram

Figure 3.5 models the maintenance request as a state machine: a request begins in the Submitted state; an admin either assigns it, moving it to Assigned, or rejects it as invalid or duplicate; an assigned request moves to In Progress once a technician begins work, can move to Blocked if parts or access are needed and back to In Progress once the blocker is resolved; a finished repair moves to Completed; and a user's confirmation moves it to the terminal Closed state, while an unsatisfied user can reopen it back to Assigned.

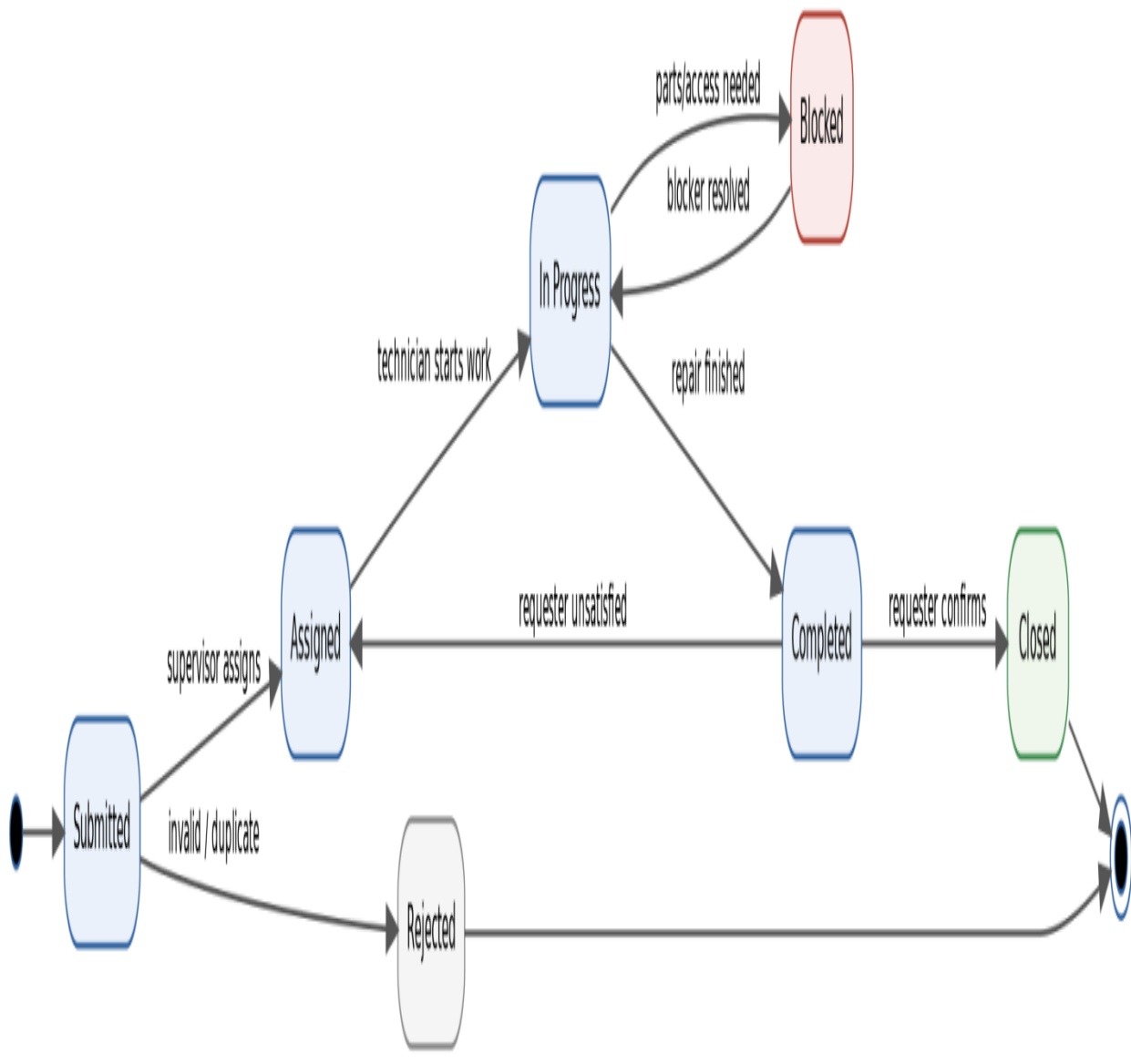


Figure 3.5: State Machine / Activity Diagram — Maintenance Request Lifecycle

3.2.3 User Interface Prototype / Wireframes

Wireframes and interactive prototypes for MSRTS (User dashboard, New Request form, Technician task list, Admin assignment board, and Admin reporting dashboard) are recommended to be built in Figma rather than as static image mockups, so that stakeholders can click through the flow before implementation begins. Figures 3.5 and 3.6 show low-fidelity wireframes for two of the highest-traffic screens, the User Dashboard and the Admin Assignment Board, produced during the analysis phase to validate the layout with stakeholders before visual design begins in Figma.

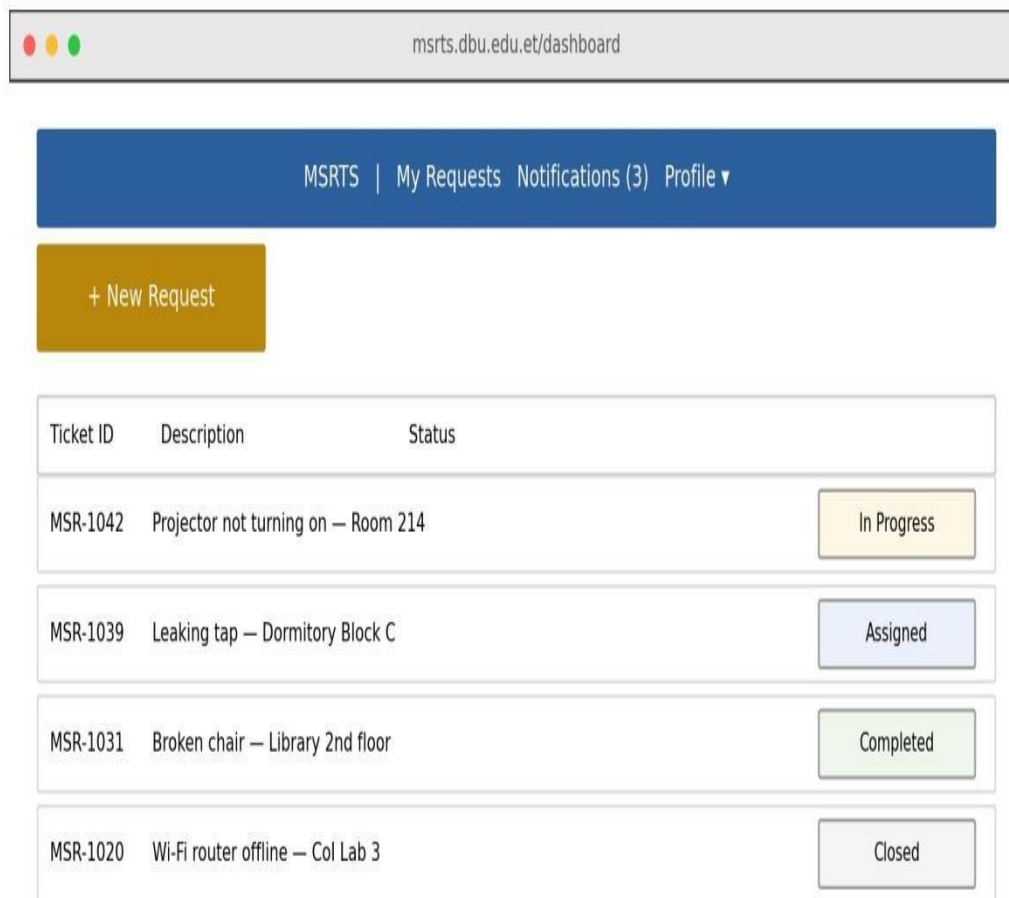


Figure 3.6: Wireframe — User Dashboard



Figure 3.7: Wireframe — Admin /admin Assignment Board (Kanban View)

The key screens to prototype in full visual fidelity are:

- ❖ Login / Registration screen.
- ❖ User Dashboard — list of 'My Requests' with status badges, and a 'New Request' button (Figure 3.6).
- ❖ New Request Form — category dropdown, description field, priority selector, location field, photo upload.
- ❖ Technician Task List — assigned requests sorted by priority/due date, with a status-update action.
- ❖ Admin Assignment Board — Kanban-style board of Submitted / Assigned / In Progress / Completed requests (Figure 3.7).
- ❖ Admin Reporting Dashboard — charts for request volume, average turnaround time, and SLA compliance.

The Figma prototype link should be inserted here once the design phase is complete:

[Figma prototype URL].

CHAPTER FOUR : SYSTEM DESIGN

4.1 Introduction

4.1.1 Purpose of the System

The purpose of the system design phase is to translate the analysis-level requirements and models from Chapter Three into a concrete architecture, database schema, API specification, and security design that the development team can implement directly. This chapter defines the software architecture, subsystem decomposition, database design, access-control mechanism, user-interface design guidelines, and deployment topology of MSRTS.

4.1.2 Design Goals

- ❖ Modularity: separate presentation, API, business logic, and data layers so each can be developed and tested independently.
- ❖ Security: enforce authentication and role-based authorization on every nonpublic endpoint.
- ❖ Maintainability: keep controllers thin and business rules in dedicated service classes, documented via OpenAPI/Swagger.
- ❖ Performance: keep common dashboard and search queries under 2 seconds by indexing frequently filtered columns (status, category, technician).
- ❖ Portability: containerize the application so it can be deployed to any Dockercompatible host.
- ❖ Testability: design service classes behind repository interfaces so business logic can be unit-tested without a live database (Section 5.2).

4.2 Current Software Architecture

As described in Chapter Two, the existing process has no software architecture: it is a manual workflow of verbal reports, phone calls, and an optional paper log book, with no application, API, or database involved. There is, consequently, no baseline system to migrate data from; MSRTS is a greenfield implementation, and the database design in Section 4.3.6 is informed by the data dictionary in Section 3.1.4.1 rather than by any existing schema.

4.3 Proposed Software Architecture

4.3.1 Overview

MSRTS is designed as a layered web application composed of a Presentation Layer (React.js client), an Application/API Layer (REST API with authentication and notification services), a Business Logic Layer (request management, assignment/SLA engine, and reporting modules), and a Data Layer (MongoDB database and object storage for attachments). Figure 4.1 shows the overall architecture.

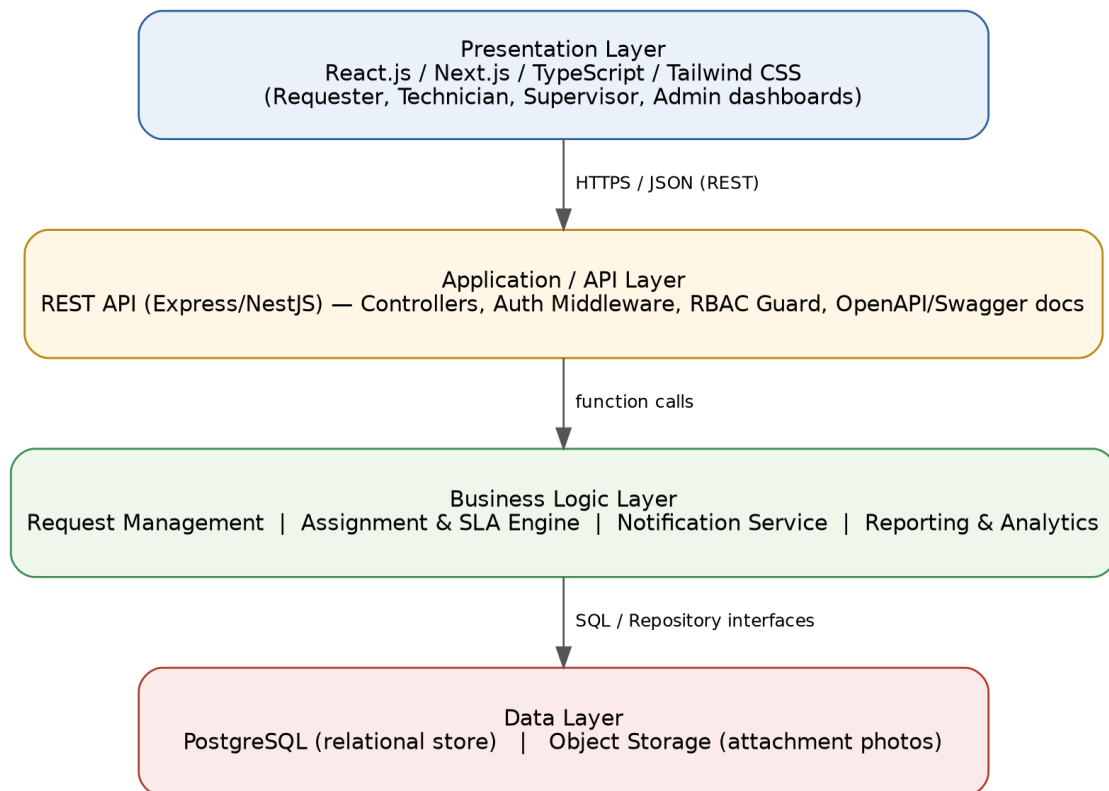


Figure 4.1: Proposed Layered Software Architecture of MSRTS

4.3.2 Architectural Pattern

The backend follows a Layered (N-Tier) Architecture with elements of Clean Architecture: HTTP controllers depend on service classes that encapsulate business rules, and service classes depend on repository interfaces that abstract database access.

This keeps business logic independent of the web framework and the ORM, which makes the system easier to test and to migrate to a different database or framework if needed. The frontend follows a component-based Model-View pattern typical of React applications, with a thin API client layer separating UI components from network calls.

4.3.3 Subsystem Decomposition

Subsystem	Responsibility
Auth Subsystem	Registration, login, JWT issuance/validation, password reset, RBAC enforcement
Request Management Subsystem	Create, read, update requests; manage categories, assets, and attachments
Assignment & SLA Subsystem	Assign technicians, track SLA deadlines, escalate overdue requests
Notification Subsystem	Send in-app and email notifications on status changes
Reporting & Analytics Subsystem	Aggregate request data and generate dashboards/exports
User & Department Management Subsystem	Manage users, roles, and organizational departments

Table 4.1: Subsystem Decomposition

4.3.4 API Design and Documentation

MSRTS exposes a RESTful API documented using the OpenAPI/Swagger specification. Table 4.2 lists the primary endpoints; the full specification will be published at `/api/docs` once implemented.

Method & Endpoint	Description	Access
POST <code>/auth/register</code>	Register a new user account	Public
POST <code>/auth/login</code>	Authenticate and receive a JWT	Public
POST <code>/requests</code>	Submit a new maintenance request	User
GET <code>/requests/{id}</code>	View a single request and its history	User (own), Technician (assigned), Admin
GET <code>/requests?status=&category=</code>	List/filter requests	Technician, Admin, Admin
POST <code>/requests/{id}/assign</code>	Assign a technician to a request	Admin, Admin
PATCH <code>/requests/{id}/status</code>	Update request status/notes	Technician (assigned), Admin
POST <code>/requests/{id}/feedback</code>	Submit rating and comment	User (own, completed request)
GET <code>/reports/summary</code>	Aggregated volume/turnaround/SLA data	Admin
GET/POST/PUT/DELETE <code>/users, /categories, /departments</code>	Manage reference/user data	Admin

Table 4.2: REST API Endpoint Summary

4.3.5 Hardware / Software Mapping

Component	Mapping
Web Client	Runs in the end user's browser (desktop or mobile); no local installation required
API Server	Node.js application packaged as a Docker container, deployed to a cloud host
Database Server	Managed MongoDB instance (cloud-hosted, e.g. Atlas, or campus server)
Object Storage	Cloud object storage (or local disk volume in campus deployment) for attachment photos

Table 4.3 lists indicative minimum specifications for the development and demo hosting environment; production sizing would be revisited once real usage data (Section 4.7) is available.

Environment	Minimum Specification
API container (per instance)	1 vCPU, 512 MB RAM — sufficient for the free-tier demo load of up to 200 concurrent users (NFR-1)
Database	1 vCPU, 1 GB RAM, 1 GB storage minimum, with automated daily backup (Section 4.8)

Client device		Any device with a modern browser (Chrome, Firefox, Edge, Safari) released within the last 3 years
---------------	--	---

Table 4.3: Hardware / Software Mapping

4.3.6 Persistent Data Management

4.3.6.1 Database Design

The database is designed as a document-oriented schema in MongoDB, using embedded documents for tightly-coupled data and reference fields for relationships between collections. Figure 4.2 shows the Document Schema diagram, and Table 3.1 (Section 3.1.4.1) lists the corresponding attributes for each entity.

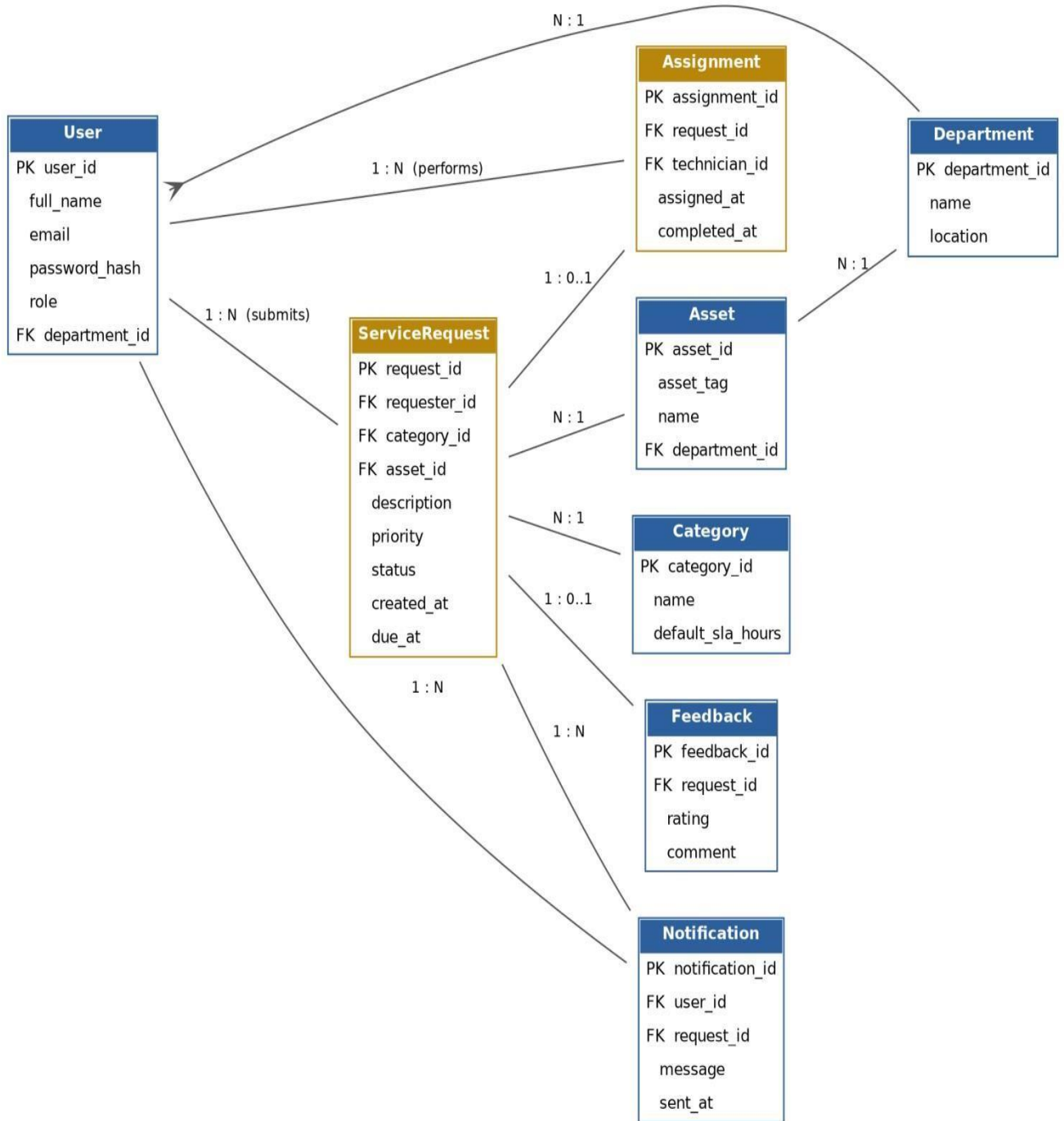


Figure 4.2: Document Schema ER-Diagram of MSRTS

Normalization notes: repeating category and department information is factored into the Category and Department tables, removing partial dependencies (2NF); no nonkey attribute depends on another non-key attribute, for example, `default_sla_hours` lives on Category rather than being copied onto every `ServiceRequest` row (3NF); and derived values such as turnaround time are computed at query time from `created_at` and `completed_at` rather than stored redundantly, to avoid update anomalies. Frequently filtered columns, `status`, `category_id`, and the assignment's `technician_id`, are indexed to satisfy the sub-two-second dashboard target in NFR-1.

4.3.7 Access Control and Security

- ❖ Authentication: JSON Web Tokens (JWT) issued at login and validated on every protected request; refresh tokens used to renew sessions without forcing frequent re-login.
- ❖ Authorization: Role-Based Access Control (RBAC) with three primary roles, User, Technician, Admin (Admin is an administrative sub-role), enforced centrally in the API middleware, not left to individual controllers.
- ❖ Data protection: all traffic served over HTTPS/TLS; passwords hashed with `bcrypt/argon2` and never logged; all user input validated and parameterized to prevent NoSQL injection and XSS; rate limiting applied to authentication endpoints to reduce brute-force risk.
- ❖ File uploads: attachment photos are validated by type and size before storage, and served from object storage rather than the application server to reduce attack surface.
- ❖ Least privilege: service accounts used by the API to reach the database are scoped to only the operations the application layer actually performs, so a compromised API credential cannot, for example, drop tables.

4.3.8 Global Software Control

MSRTS uses an event-driven control style at the boundary between subsystems: a state change in the Request Management Subsystem (e.g., status set to 'Assigned') publishes an internal event that the Notification Subsystem subscribes to, decoupling notification delivery from the core request-update transaction. Within a single request/response cycle, control follows a straightforward top-down call chain:

Controller to Service to Repository to Database.

4.3.9 Boundary Conditions

□ *Startup*

On startup, the API server verifies database connectivity and runs pending migrations before accepting traffic.

□ *Shutdown*

On a graceful shutdown signal, the server stops accepting new connections, allows inflight requests to complete, and closes the database connection pool.

□ *Error / Failure Handling*

- ❖ If the database is unreachable, the API returns a 503 Service Unavailable response with a generic error message and logs the incident for admins.
- ❖ If notification delivery (email/SMS) fails, the request status update itself still succeeds, and the failed notification is retried asynchronously rather than blocking the user's action.
- ❖ Client-side, network failures during submission are caught and the user is shown a retry option rather than losing the entered form data.

4.4 User Interface Design Guidelines

Although the detailed visual design is produced in Figma per Section 3.2.3, the following guidelines apply consistently across all MSRTS screens so that the implemented UI matches the wireframes and satisfies NFR-8 (accessibility) and NFR10 (responsiveness):

- ❖ **Color:** a primary navy/blue tone signals brand and primary actions, a gold/amber accent marks warnings and pending states, and status badges use consistent colors across every screen (e.g., amber for 'In Progress', green for 'Completed', grey for 'Closed'), matching the palette used in Figures 3.5 and 3.6.
- ❖ **Typography:** a single system/sans-serif font family is used throughout, with no more than three font sizes per screen, to keep dashboards scannable.
- ❖ **Layout:** dashboards use a persistent top navigation bar and a single primary call-to-action per screen (e.g., 'New Request'), avoiding competing actions.

- ❖ Accessibility: all interactive elements are reachable by keyboard, all status information conveyed by color is also conveyed by text or icon (not color alone), and color contrast meets WCAG 2.1 AA (NFR-8).
- ❖ Responsiveness: layouts collapse from multi-column (desktop) to singlecolumn (mobile) at defined breakpoints, tested down to 360px width (NFR-10).
- ❖ Feedback: every user action (submit, assign, status update) produces an immediate, visible confirmation or error message (NFR-12), rather than a silent state change.

4.5 Component Diagram

The system decomposes into the following deployable/logical components, matching the subsystems in Section 4.3.3: a Web Client component; an API Gateway/Controller component; Auth, Request Management, Assignment & SLA, Notification, and Reporting service components; and a Data Access component that all services depend on to reach the MongoDB database and object storage. Figure 4.3 shows these components and their dependencies, which flow strictly downward, the Web Client depends on the API Gateway, the API Gateway depends on the service components, and the service components depend on the Data Access component, so no lower layer depends on a higher one, matching the layered architecture shown in Figure 4.1.

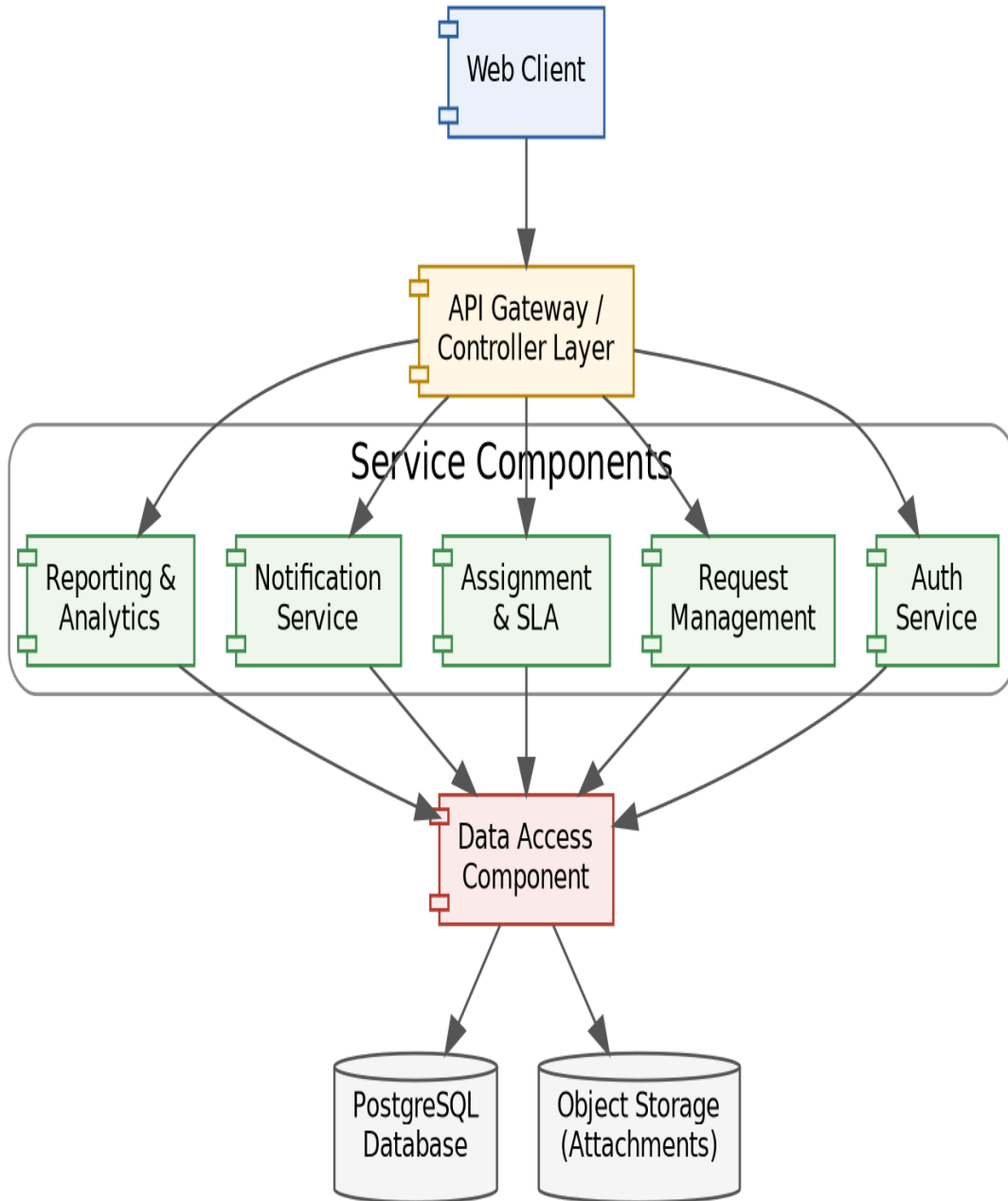


Figure 4.3: Component Diagram of MSRTS

4.6 Deployment Diagram

Figure 4.4 shows the deployment view of MSRTS. The static frontend is hosted on a CDN/static host (e.g., Vercel); the API is packaged as Docker containers behind a load balancer/reverse proxy for horizontal scalability; the database is a managed MongoDB instance; and outbound email/SMS notifications are sent through a thirdparty gateway API. This containerized deployment allows the

same image to be run on any Docker-compatible host, including free-tier cloud platforms (Render) or, later, on-premises university servers.

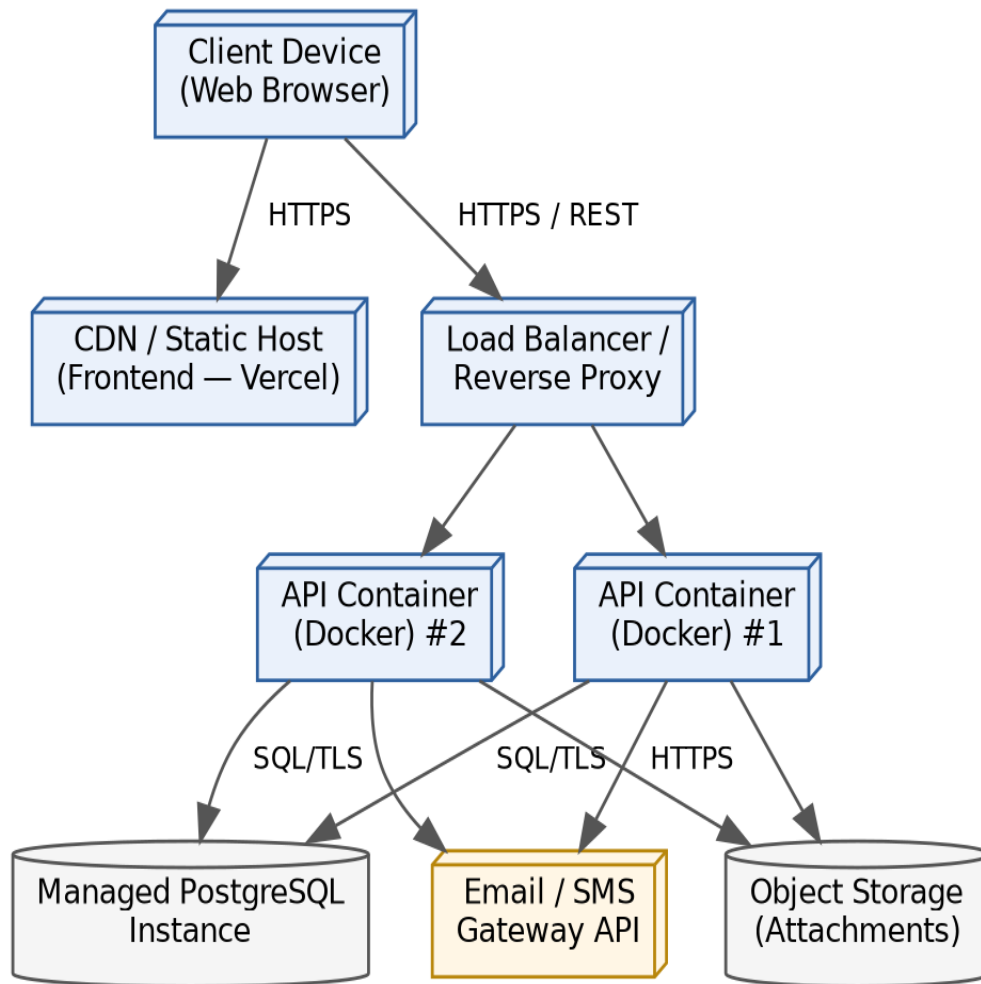


Figure 4.4: Deployment Diagram of MSRTS

4.7 Performance and Scalability Considerations

The layered, stateless design of the API (Section 4.3.2) means additional API containers can be added behind the load balancer in Figure 4.4 without code changes, directly satisfying NFR-2. The indexing strategy described in Section 4.3.6.1 keeps the dashboard and search queries referenced in NFR-1 fast as request volume grows. Because attachments are served from object storage rather than the application server (Section 4.3.7), large photo uploads do not compete with API request-handling capacity. If concurrent usage significantly exceeds the 200-user target in NFR-1, the next scaling step would be to introduce a read replica for the reporting subsystem (Section 4.3.3), so that

heavy analytics queries in Section 2.10.1 (FR-13) do not compete with the transactional request-submission and status-update workload.

4.8 Backup and Disaster Recovery

- ❖ The MongoDB database is backed up automatically on a daily schedule, with backups retained for at least 30 days, satisfying the 24-month status-history retention in NFR-14 through a combination of live data and archived backups.
- ❖ Object storage (attachment photos) is configured with versioning where the hosting provider supports it, so an accidental overwrite or deletion can be recovered.
- ❖ A restore procedure is documented and tested at least once before go-live, so that the first time a restore is attempted is not during an actual incident.
- ❖ Application configuration (environment variables, container images) is stored in version control and the deployment pipeline, so the API layer itself can be redeployed from scratch without manual reconstruction.

CHAPTER FIVE :TESTING AND QUALITY ASSURANCE

5.1 Testing Objectives

The purpose of testing MSRTS is to verify that every functional requirement in Section 2.10.1 behaves as specified, that every non-functional requirement in Section 2.10.2 is met under realistic conditions, and that the system is safe to place in front of real users, technicians, and admins during User Acceptance Testing. Testing is treated as a continuous activity carried out within each Agile sprint (Section 1.11.2), not as a single phase at the end of the project.

5.2 Levels of Testing

5.2.1 Unit Testing

Individual functions, service classes, and React components are tested in isolation using Jest. Because the business logic layer is separated from the web framework and the database behind repository interfaces (Section 4.3.2), service-level unit tests can run against in-memory fakes rather than a live database, keeping the suite fast enough to run on every commit.

5.2.2 Integration Testing

API endpoints are tested against a real (test) MongoDB instance using Postman/Newman collections, verifying that each endpoint in Table 4.2 returns the documented status codes and payload shape, and that RBAC rules (Section 4.3.7) correctly reject requests from a role that should not be able to perform the action.

5.2.3 End-to-End (E2E) Testing

Critical user journeys, submit a request, assign a technician, update status through to completion, leave feedback, and generate a report, are scripted as automated endtoend tests that exercise the full stack from the browser down to the database, so that a regression in any layer is caught before release.

5.2.4 User Acceptance Testing (UAT)

Before final deployment, a small group of real users and maintenance staff, recruited from the same interviews described in Section 1.11.1, are asked to complete a set of representative tasks (submit a request,

assign a technician, close a request) without guidance beyond the in-app interface. UAT feedback is used to validate NFR-11 (a first-time user can submit a request without training) directly, rather than by inference from developer testing alone.

5.3 Test Case Design

Table 5.1 lists a representative sample of test cases, one or two per major functional requirement, covering both the expected (happy) path and at least one failure path. The full test suite maintained in the project repository expands this sample to cover every functional and non-functional requirement listed in Chapter Two.

Test ID	Requirement	Input / Steps	Expected Result
TC-01	FR-1	Register with a valid university email and a strong password	Account created; confirmation shown; user can log in
TC-02	FR-1	Attempt login with an incorrect password, 6 times in a row	Account temporarily rate-limited after repeated failures (NFR4)
TC-03	FR-2, FR-3	Submit a request with category, description, priority, location, and a valid photo	Request created with status 'Submitted' and a unique tracking ID
TC-04	FR-2	Submit a request with the description field left blank	Validation error shown; no request record created
TC-05	FR-4	User opens 'My Requests' after an admin changes a request's status	Updated status is visible without a page reload delay beyond NFR-1
TC-06	FR-10	Admin assigns a technician whose specialty matches the request category	Status becomes 'Assigned'; technician and user notified

TC-07	FR-10	Admin attempts to assign a request to a technician outside the matching category	System allows override but flags the mismatch for the audit log (FR- 14)
TC-08	FR-8	Technician updates an assigned request to 'Blocked' with a note	Status becomes 'Blocked'; admin is notified
TC-09	FR-12	Admin sets a 24-hour SLA for the 'Electrical' category; a request in that category passes 20 hours open	Request is flagged as approaching SLA breach; admin alerted
TC-10	FR-13	Admin exports a monthly report as CSV	Downloaded file contains volume, turnaround, and category breakdown matching the on-screen dashboard
TC-11	NFR-5	A Technician account attempts to call POST /users (admin-only)	API returns 403 Forbidden; action is logged
TC-12	NFR-6	Inspect the users table directly in the database	password_hash column contains only bcrypt/argon2 hashes, never plain text
TC-13	NFR-1	Load-test the dashboard endpoint with 200 concurrent simulated users	95th-percentile response time remains under 2 seconds
TC-14	NFR-8	Run an automated accessibility audit (e.g., axe) against the User Dashboard	No critical WCAG 2.1 AA violations reported

Table 5.1: Sample Test Cases

5.4 Test Environment

Automated unit and integration tests run in a continuous-integration pipeline triggered on every push to the shared GitHub repository (Section 1.11.4), against an isolated test database seeded with realistic sample data derived from the interview and questionnaire findings in Section 1.11.1. UAT

is conducted against a separate staging deployment that mirrors the production configuration described in Section 4.6, so that issues found during UAT reflect real deployment conditions rather than a developer's local machine.

5.5 Defect Management Process

Defects found during any level of testing are logged in the same project-management tool used for sprint planning (Section 1.11.6), tagged with severity (Blocker, Major, Minor, Cosmetic) and the requirement ID they relate to, so that defect trends can be cross-referenced with the Requirement Traceability Matrix in Appendix E. Blocker and Major defects are triaged before the next sprint review; Minor and Cosmetic defects are scheduled into the backlog.

CHAPTER SIX : CONCLUSION AND FUTURE WORK

6.1 Summary

This document has presented the Software Requirements Specification for the Maintenance Service Request and Tracking System (MSRTS), covering the background and motivation for the project, a detailed description of the existing manual maintenance-reporting process at Debre Berhan University and its performance, security, scalability, and user-experience bottlenecks, the functional and non-functional requirements of the proposed system, the analysis-level use case and object models, the dynamic behavioral models, and the proposed system design, spanning software architecture, database design, API specification, security design, user-interface guidelines, and deployment topology.

Taken together, these chapters establish that MSRTS is both a feasible and a wellscoped response to the problem stated in Section 1.5: it replaces an undocumented, phone-and-paper process with a structured, auditable, and measurable workflow, using a technology stack and architecture (Chapter Four) that the project team can realistically deliver within the four-week accelerated schedule set out in Section 1.13.

6.2 Limitations of the Current Version

Consistent with the scope boundaries defined in Section 1.8, the version of MSRTS specified in this document does not attempt automated technician assignment, native mobile applications, IoT-based automatic fault detection, offline operation, or multilanguage support. These exclusions were deliberate scope decisions made to keep the project deliverable within the available time and team size, not oversights, and each is revisited below as a candidate for future work.

6.3 Future Enhancements

- ❖ Automated, rules- or ML-based technician assignment that suggests, or directly assigns, the best-matched available technician instead of relying entirely on admin judgement.
- ❖ A native or installable mobile application (or a Progressive Web App) so technicians and users can receive push notifications and work with partial offline support in areas of campus with weak connectivity.
- ❖ IoT-based automatic fault detection for high-value or safety-critical assets (e.g., server-room temperature or water-leak sensors) that can auto-generate a request without waiting for a human to notice.

- ❖ Integration with a procurement or ERP system so that a request requiring spare parts can trigger a purchase workflow instead of leaving procurement outside the system's visibility.
- ❖ Multi-language support (at minimum, Amharic) to reduce friction for users and technicians who are more comfortable outside English.
- ❖ A knowledge base of common faults and their resolutions, surfaced to technicians when they open a request in a matching category, to shorten repeatfault resolution time.

Preventive-maintenance scheduling, using the asset failure history captured in the Asset and ServiceRequest tables (Section 3.1.4.1) to proactively schedule inspections before a predictable failure occurs.

REFERENCES

- Fielding, R. T. (2000). *Architectural styles and the design of network-based software*.
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- IEEE. (1998). *IEEE recommended practice for software requirements specifications*.
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- MongoDB, Inc. (2024). *MongoDB documentation*. [MongoDB Documentation](#)
- Object Management Group. (2017). *OMG Unified Modeling Language (UML) (Version 2.5.1)*. [OMG UML Specification](#)
- OWASP Foundation. (2021). *OWASP Top 10: Web application security risks*. [OWASP Top 10](#)
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.
- Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. Scrum.org. [The Scrum Guide](#)
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. [WCAG 2.1](#)

APPENDIX A: SAMPLE INTERVIEW QUESTIONS

The following questions were used, or are representative of the questions used, during the semi-structured interviews with facilities-office staff and users referenced in Section 1.11.1.

A.1 Questions for Facilities / Maintenance Office Staff

- ❖ How is a maintenance problem currently reported to your office (phone, in person, written note)?
- ❖ Is any record kept of a request once it is received? If so, in what form?

- ❖ How do you decide which technician handles a given request?
- ❖ How do you currently prioritize urgent requests over routine ones?
- ❖ How would you currently find out how long a particular type of repair typically takes?
- ❖ What is the most common source of delay between a request being reported and being resolved?
- ❖ How do you currently know whether a repair was completed to the user's satisfaction?

A.2 Questions for Users (Students and Staff)

- ❖ Think of the last maintenance problem you reported. How did you report it, and to whom?
- ❖ How did you find out whether, or when, it had been resolved?
- ❖ Did you have to follow up more than once? If so, how many times?
- ❖ What would have made that experience easier?
- ❖ Would you be comfortable reporting a maintenance issue through a web form on your phone or laptop?

APPENDIX B : SAMPLE QUESTIONNAIRE

The following short questionnaire was distributed to a sample of staff and students to quantify how often maintenance problems are reported and how satisfied respondents are with current response times, as referenced in Section 1.11.1.

No.	Question	Response Type
1	How often do you personally report a maintenance issue (e.g., broken equipment, plumbing, electrical fault)?	Never / Rarely / Monthly / Weekly or more
2	How do you currently report a maintenance issue?	Phone / In person / Written note / Other

3	On average, how long does it take for a reported issue to be resolved?	Same day / 2-3 days / Over a week / Don't know
4	How satisfied are you with the current process?	1 (Very dissatisfied) to 5 (Very satisfied)
5	Have you ever reported an issue that you believe was never addressed?	Yes / No / Not sure
6	Would you use a web-based system to report and track maintenance issues if one were available?	Yes / No / Maybe
7	Any additional comments about the current maintenance-reporting process?	Open text

APPENDIX C SAMPLE API REQUEST / RESPONSE PAYLOADS

This appendix provides illustrative JSON payloads for the two highest-traffic endpoints listed in Table 4.2, so that frontend and backend developers can align on the exact contract before implementation. These examples are illustrative; the authoritative contract is the OpenAPI/Swagger specification referenced in Section 4.3.4.

C.1 POST /requests (Submit Maintenance Request — FR-2, FR-3) Request

```
POST /api/v1/requests
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "category_id": "cat_electrical",
  "asset_id": "asset_proj_214",
  "description": "Projector in Room 214 does not power on.",
  "priority": "high",
  "location": "College of Informatics, Room 214",
```

```
"photo_url": "https://storage.msrts.dbu.edu.et/uploads/abc123.jpg" }
```

Response — 201 Created

```
{
  "request_id": "MSR-1042",
  "status": "Submitted",
  "created_at": "2026-08-07T09:14:00Z",
  "due_at": "2026-08-08T09:14:00Z"
}
```

C.2 POST /requests/{id}/assign (Assign Technician — FR-10)

Request

POST /api/v1/requests/MSR-1042/assign

Authorization: Bearer <jwt>

Content-Type: application/json

```
{
  "technician_id": "tech_0231" }
```

Response — 200 OK

```
{
  "request_id": "MSR-1042",
  "status": "Assigned",
  "assignment": {
    "technician_id": "tech_0231",
    "assigned_at": "2026-08-07T10:02:00Z"
  }
}
```

Response — 403 Forbidden (RBAC violation, NFR-5)

```
{
  "error": "forbidden",
  "message": "Only an Admin may assign a technician." }
```

APPENDIX D GLOSSARY OF TERMS

Term	Definition
Asset	A piece of equipment or fixture (e.g., a projector, a water pump) that can be the subject of a maintenance request.
Category	A classification of maintenance issue (electrical, plumbing, ICT, carpentry, general) used for routing and SLA configuration.
MSRTS	Maintenance Service Request and Tracking System, the system specified in this document.
Priority	A user- or admin-set indicator of urgency (e.g., low, medium, high, urgent) attached to a request.
User	Any student, faculty, or staff member who submits a maintenance request.
RBAC	Role-Based Access Control; an authorization model that grants permissions based on a user's assigned role.
SLA	Service-Level Agreement; here, a target maximum resolution time configured per category.
Admin	A user role, an administrative sub-role, responsible for assigning technicians and monitoring the request queue.
Technician	A user role responsible for executing and updating the status of assigned maintenance requests.

Ticket / Request	A single structured record representing one reported maintenance problem, from submission to closure.
------------------	---

APPENDIX E REQUIREMENT TRACEABILITY MATRIX

Table E.1 traces each functional requirement in Section 2.10.1 to the use case(s) in Section 3.1.3 that realize it and the sample test case(s) in Section 5.3 that verify it, so that coverage gaps are visible at a glance. A dash indicates that the corresponding use case or test case was not included in the illustrative samples in this document and should be added to the full project test suite.

Req. ID	Requirement (summary)	Use Case(s)	Test Case(s)
FR-1	Register / log in securely	UC-1	TC-01, TC-02
FR-2	Submit request with details	UC-1	TC-03, TC-04
FR-3	Unique tracking ID on submission	UC-1	TC-03
FR-4	View real-time status	UC-1	TC-05
FR-5	Status-change notifications	UC-2, UC-3	TC-06, TC-08
FR-6	Rate / feedback on completed request	-	-

FR-7	Technician task list	UC-3	-
FR-8	Technician status update + notes	UC-3	TC-08
FR-9	Completion triggers notification	UC-3	-
FR-10	Admin assignment	UC-2	TC-06, TC-07
FR-11	Admin management of reference data	-	TC-11
FR-12	SLA configuration and alerts	UC-5	TC-09
FR-13	Reports on volume/turnaround	UC-4	TC-10
FR-14	Audit log of key actions	UC-2, UC-3	TC-07

Table E.1: Requirement Traceability Matrix